

Amojilar, Rogell B.
Dela Cruz, Maria Kylee A.
Gonzalez, Rheanna V.
Manzano, Elohim Lee L.
Oleriana, Mico Laurenz D.
San Pedro, Jerome M.
Villanueva, Mikaela Marie A.

INTRODUCTION

Java has been a popular programming language for over 29 years and billions of devices run java. It can work on different platforms such as Windows, Mac, Linux, Raspberry Pi, etc. Java was created by James Gosling in 1995 and now it is owned by Oracle. Furthermore, Java is essentially a programming language that is object-oriented, which means that it expresses real-world entities as objects. Not only object-oriented but also a strong and adaptable programming language and to write Java code on one computer and run it on any device that has Java installed because of Java Virtual Machine (JVM). Many fields used Java for their systems because of the reliability, simplicity and scalability of this program.

Additionally, this project aims to have every aspect of the Java program may benefit to every user and flexible system. Our system was based on business and accounting wherein this system is a must, specifically it has more complex data that may conflict with the database of the company or store. The project will use different data structures and provide understandable instructions for the project's aim of being user-friendly.

Nonetheless, this will focus on creating a system that aims to have different data structures for efficient use for businesses and accountancy. This project used six (6) different data structures and algorithms including this following:

- **Array & Pointers** - The array and pointers correspond to data types with many distinctions and similarities. An array is a data structure representing a collection of elements of the exact same kind stored in consecutive memory areas. A pointer is a variable that stores the memory addresses of another variable (GeeksforGeeks, 2023). In some circumstances, we can use pointers instead of arrays, and arrays are automatically converted to pointers when supplied to a function.

- **Linked Lists** - The LinkedList is a versatile data structure that is especially useful when constant-time insertions and deletions are required. Because a LinkedList functions as a dynamic array and we do not have to specify the size when creating it, the list's size increases on its own when we dynamically add and remove items. A doubly linked list has an extra pointer, commonly referred to as the previous pointer, in addition to the next pointer and data found in a singly linked list (GeeksforGeeks, 2024).
- **Stack** - Stack is a linear data structure that operates on the LIFO (Last In First Out) principle, which states that the last element inserted is the first to be pushed out. The Stack class extends Vector and adds features related to stack operations including push, pop, peek, empty, and search (GeeksforGeeks, 2024). It signifies that both insertion and deletion take place at one end only.
- **Queue** - A Queue Data Structure is a fundamental notion in computer science that stores and manages data in a predefined order. It operates on the "First in, First out" (FIFO) principle, which states that the first element added to the queue is also the first to be deleted (GeeksforGeeks, 2024).
- **Search & Sorting**- We can do simple operations on arrays, such as sorting and searching. A Sorting Algorithm rearranges an array or list of components in a specific order. Most computer language libraries have sorting functionality (GeeksforGeeks, 2024).
- **Hash Tables**- Hashtable is a historical Java class for storing key-value pairs, but it is still commonly used in certain contexts. The Hashtable class creates a hash table that maps keys to values. Any non-null object may be used as a key or value (GeeksforGeeks, 2024).

Background of the Project

This group project focuses on the **Business and Accounting** category, showcasing a Java-based product data management system designed to demonstrate the practical use of fundamental data structures and algorithms. The system is interactive, enabling users to input commands and receive real-time outputs based on their queries. It manages product data efficiently, utilizing various coding techniques such as queues for FIFO operations, stacks for LIFO processing, search algorithms for locating data quickly, sorting techniques for organizing records, and storage structures like arrays, ArrayLists, and LinkedLists. This project highlights the versatility and functionality of these programming concepts in real-world applications.

Basic Features and Functionalities

1. Login System

- At the start of the program, a login system ensures security and proper user authentication.
- The user must provide a valid username and password that matches the records stored in an array.
- Only after successful authentication can the user proceed to access the main menu and system features.

2. Main Menu

The main menu serves as the central hub for navigating the program. It offers various options, including exploring different data structures and algorithms implemented in the system.

3. Group Members

The main menu includes a "Group Members" option, allowing users to view information about the team behind the program. Users can also exit the system directly from this menu if needed.

4. Data Structure and Algorithms:

- **Array & Pointers**- creation and manipulation of product data record
- **Linked Lists** - Allow dynamic addition and removal of product data without a fixed size. The system provides options for singly linked lists and doubly linked lists, supporting operations like adding, removing, and displaying product data in ways that align with user preferences.

- **Stack** - Implements the Last In, First Out (LIFO) method with essential operations such as push, pop, and peek. It manages recent actions, like undoing changes, and allows users to display updated product data and view the current stack size.
- **Queue** - Offers three implementation options: arrays, linked lists, and standard queue operations. It efficiently organizes product data in a sequential First In, First Out (FIFO) order.
- **Search & Sorting** - Effortlessly find and organize product records using three search methods: linear search, binary search, and interpolation search. For sorting, the system arranges product data based on user preferences, with options like bubble sort, selection sort, and insertion sort to organize the records as desired.
- **Hash Tables** - The module lets users add, view, and delete product data using hash functions. It tracks collisions, provides insights into their handling, and offers sample applications for efficient product record management through a simple menu-driven interface.

5. Invalid Inputs

In the event of an invalid input, the system will detect the error and display a clear error message to inform the user of the issue. Following this, the system will prompt the user to enter the correct input again,

Expected Behavior Upon Execution

1. The user is prompted to log in with valid credentials.
2. After successful login, the main menu appears with various data structure options.
3. The user selects an option by entering the corresponding number.
4. A submenu appears for actions such as adding, viewing, or deleting product data.
5. If invalid input is entered, an error message is shown, and the user is prompted to enter valid data again.
6. The process continues until the user decides to exit the system.

Screen Output (Design)

```
Log In
-----
UserID: 2023100279
Password: password
Logged In...
```

Figure 1.0 - Login Display

Upon starting the program, this is what the users will see. They will be asked to input their credentials, and if those credentials match with those in our system, they will be able to login.

```
MAIN MENU
-----
[1]Array & Pointers
[2]Linked Lists
[3]Stack Data Structure
[4]Queue Data Structure
[5]Search & Sorting Algorithms
[6]Hash Tables
[7]Group Members
[0]Exit
Enter your Choice ->
```

Figure 1.1 - Main Menu

Users that have logged in successfully will be presented with the main menu, where it displays all available methods for users to choose from.

ARRAY & POINTERS:

- [1] One Dimensional Array
- [2] Two Dimensional Array
- [3] Sample Applications
- [0] Return to Main Menu

Figure 1.2 - Array Main Display

Menu that displays upon choosing arrays and pointers.

- 1
- [1] Add
- [2] Edit
- [3] Delete
- [4] Display Item
- [0] Back to Array Main Menu

Figure 1.3 - Array Display 1

Menu that contains the functions of 1D Array.

TWO DIMENSIONAL ARRAY

- [1] Add
- [2] Edit
- [3] Delete
- [4] Display Item
- [0] Back to Array Main Menu

4

DISPLAY ITEMS

- [1] Display Item in Array
- [2] Display Item in Organized
- [3] Back to 2-D Main Menu

Figure 1.4 - Array Display 2

Menu that contains the functions of 2D Array.

3

- [1] Tic Tac Toe
- [2] Connect Four
- [0] Back to Main Menu

Figure 1.5 - Array Display 3

Sample applications menu of array & pointers.

LINKED LIST DATA STRUCTURE

```
[1] Singly Linked List
[2] Doubly Linked List
[3] Linked List Class in Java
[0] Back to Main Menu
Enter your Choice -> 1
```

Figure 2.0 - Linked List Main Display

Menu that displays upon choosing linked list data structures.

Singly Linked List

Inventory Management System

```
1. Add Product
2. Display Products
3. Remove a Product
0. Back to Linked List Menu
Choose an action: 0
```

Figure 2.1 - Linked List Display 1

Menu that contains the functions of our singly linked list.

Doubly Linked List Menu

Inventory Management System

1. Add Product
2. Display Products
3. Display Products in Reversed
4. Remove a Product
0. Back to Linked List Menu

Choose an action:

Figure 2.2 - Linked List Display 2

Menu that contains the functions of our doubly linked list.

Linked List in Class Menu

Inventory Management System

1. Add Product
2. Display Products
3. Remove a Product
0. Back to Linked List Menu

Choose an action:

Figure 2.3 - Linked List Display 3

Menu that contains the functions of our linked list class in java.

STACK DATA STRUCTURE

```
--=====
[1] Stack using Array
[2] Stack using Linked List
[3] Stack Class in Java
[0] Back to Main Menu
Enter your Choice ->
```

Figure 3.0 - Stack Main Display

Menu that displays upon choosing stack data structures.

```
Enter your Choice -> 1
```

```
Array Size: 5
```

Stack Array

```
--=====
[1] Push Product
[2] Pop Product
[3] Peek Product
[4] Display Product/s
[5] List Size
[0] Back to Stack Data Structure
Enter your Choice -> 0
```

Figure 3.1 - Stack Display 1

Menu that contains the functions of stack using array.

Stack Linked List

```
-----  
[1] Push Product  
[2] Pop Product  
[3] Peek Product  
[4] Display Product/s  
[5] List Size  
[0] Back to Stack Data Structure  
Enter your Choice -> 0
```

Figure 3.2 - Stack Display 2

Menu that contains the functions of stack using linked list.

Stack In Class

```
-----  
[1] Push Product  
[2] Pop Product  
[3] Peek Product  
[4] Display Product/s  
[5] List Size  
[0] Back to Stack Data Structure
```

Figure 3.3 - Stack Display 3

Menu that contains the functions of stack in java class.

QUEUE DATA STRUCTURE

```
-----  
[1] Queue using Array  
[2] Queue using Linked List  
[3] Queue Class in Java  
[0] Back to Main Menu  
Enter your Choice ->
```

Figure 4.0 - Queue Main Display

Menu that displays upon choosing queue data structure.

```
Enter your Choice -> 1  
Array Size: 10  
  
Queue Array  
-----  
[1] Enqueue Product  
[2] Dequeue Product  
[3] Peek Product  
[4] Display Product/s  
[5] List Size  
[0] Back to Queue Data Structure  
Enter your Choice -> 0
```

Figure 4.1 - Queue Display 1

Menu that contains the functions of the queue using arrays.

Queue Linked List:

- [1] Enqueue Product
- [2] Dequeue Product
- [3] Peek Product
- [4] Display Product/s
- [5] List Size
- [0] Back to Queue Data Structure

Enter your Choice ->

Figure 4.2 - Queue Display 2

Menu that contains the functions of the queue using linked lists.

Queue In Class

- [1] Enqueue Product
- [2] Dequeue Product
- [3] Peek Product
- [4] Display Product/s
- [5] List Size
- [0] Back to Queue Data Structure

Enter your Choice ->

Figure 4.3 - Queue Display 3

Menu that contains the functions of the queue using the java class.

Search And Sort Menu

```
-----  
[1] Search Algorithms  
[2] Sorting Algorithms  
[3] Sample Applications  
[0] Back to Main Menu  
Enter your Choice ->
```

Figure 5.0 - Search & Sort Main Display
Menu that displays upon choosing search and sort.

Search Algorithms

```
-----  
[1] Linear Search  
[2] Binary Search  
[3] Interpolation Search  
[4] Display Sample Records  
[0] Back to Search and Sort Menu  
Enter your Choice -> 0
```

Figure 5.1 - Search & Sort Display 1
Menu that contains the different types of search algorithms.

Sort Algorithms

```
-----  
[1] Sort from Highest to Lowest Price  
[2] Sort from Lowest to Highest Price  
[3] Display Sample Records  
[0] Back to Search and Sort Menu  
Enter your Choice -> 3
```

Figure 5.2 - Search & Sort Display 2

Menu that contains options for sorting the sample records.

```
Enter your Choice -> 6  
Size: 5  
  
HASH TABLES  
-----  
[1] Hash Function  
[2] Hash Collision  
[3] Sample Applications  
[0] Back to Main Menu  
Enter your Choice -> 1  
  
Hash Table  
-----  
[1] Add Product  
[2] View Product  
[3] Delete Product  
[0] Back to Queue Data Structure
```

Figure 6.0 - Hash Tables Display

Menu that displays upon choosing hash tables, different functionalities of hash.

```
GROUP MEMBERS
-----
[1] Group Leader: Oleriana, Mico Laurenz D.
[2] Member 1: Amojilar, Rogell B.
[3] Member 2: Dela Cruz, Maria Kylee A.
[4] Member 3: Gonzalez, Rheanna V.
[5] Member 4: Manzano, Elohim Lee L.
[6] Member 5: San Pedro, Jerome M.
[7] Member 6: Villanueva, Mikaela Marie A.
[0] Back to Main Menu
Enter your Choice -> 1

Name: Oleriana, Mico Laurenz D.
Student No.: 2023100190
Age: 19
Address: Bocaue, Bulacan
```

Figure 7.0 - Member Menu Display

Menu that displays all the members, picking a member then displays their full name, student number, age, and address.

Program Listing (Java Source Code)

```
startMenu
import java.util.*;

public class startMenu {
    public static final String RESET =
"\033[0m";
    public static final String GREEN =
"\u001B[32m";
    public static final String
GREEN_BOLD = "\033[1;32m";
    public static final String RED_BOLD =
"\033[1;31m";
    public static final String
YELLOW_BOLD = "\033[1;33m";
    public static Scanner sc = new
Scanner(System.in);
    public static String user, pass;

    public static boolean loop = true, lock
= false;
    public static int hold; // for holding the
value of the index
    public static void openMenu(){
        String[] studID =
{"2023100190","2023100385","2023100
452","2023100377","2023100279","2023
100179","2023100341","EdwinGarcia"};
        // array of userIDs
        String[] pw =
{"jz0921","1124","kahitano","040706","pa
ssword","jj2710","101004","godspeed"};
        // array of corresponding passwords
        while (loop){
```

```

System.out.println(GREEN_BOLD +
"\nLog In" + RESET);

System.out.println("-----
-----");
        System.out.print(GREEN +
"UserID: " + RESET);
        user = sc.next();
        for(int x = 0; x <= studID.length -
1; x++){
            if (user.equals(studID[x])){
                lock = true;
                hold = x;
                break;
            }
        }
        if(lock) {
            System.out.print(GREEN +
"Password: " + RESET);
            pass = sc.next();
            if(pass.equals(pw[hold])){
                System.out.println("Logged
In...");

                mainMenu.openMenu();
            }else{
                System.out.println("UserID
and Password does not match.");
            }
        }else{

System.out.println(RED_BOLD + "No
Records for UserID: " + RESET + user);
        }
    }
}
public static void main(String[] args) {
    openMenu();
}
}

```

ArrayQueue

```

public class ArrayQueue {
    private String[][] queue;
    private int front;
    private int rear;
    private int size;
    private int capacity;
    // Constructor to initialize queue
    public ArrayQueue(int capacity) {
        this.capacity = capacity;
        queue = new String[capacity][4];
        front = 0;
        rear = -1;
        size = 0;
    }
    // Check if the queue is full
    public boolean isFull() {
        return size == capacity;
    }
    // Check if the queue is empty
    public boolean isEmpty() {
        return size == 0;
    }
    // Enqueue operation to add an
    element at the rear
    public void enqueue(String ITMNO,
String QTY, String UPRICE, String
DISC, String ITMDESC) {
        Product product = new
Product(ITMNO, QTY, UPRICE, DISC,
ITMDESC);
        if (isFull()) {

System.out.println(startMenu.RED_BOL
D + "Queue is full! Cannot enqueue " +
startMenu.RESET + product);
            return;
        }
        rear = (rear + 1) % capacity;
    }
}

```

```

        queue[rear] = new String[]{ITMNO,
ITMDESC, QTY, UPRICE, DISC};
        size++;

```

```

System.out.println(startMenu.GREEN +
"Enqueued: " + startMenu.RESET +
product);
}

```

```

// Dequeue operation to remove an
element from the front

```

```

    public void dequeue() {
        if (isEmpty()) {

```

```

System.out.println(startMenu.RED_BOLD + "Queue is empty! Cannot dequeue."
+ startMenu.RESET);
        return;
    }

```

```

System.out.println(startMenu.GREEN_BOLD + "Dequeued:\n" +
startMenu.GREEN + "Item Code No.: "
+ startMenu.RESET + queue[front][0] +
" | " + startMenu.GREEN + "Item
Description: " + startMenu.RESET +
queue[front][1] + " | " +
startMenu.GREEN + "Quantity: " +
startMenu.RESET + queue[front][2] + " |
" + startMenu.GREEN + "Unit Price: " +
startMenu.RESET + queue[front][3] + " |
" + startMenu.GREEN + "Discount: " +
startMenu.RESET + queue[front][4]);
        front = (front + 1) % capacity;
        size--;
    }

```

```

// Peek operation to see the front
element

```

```

    public void peek() {
        if (isEmpty()) {

```

```

System.out.println(startMenu.RED_BOLD + "Queue is empty! No front element."
+ startMenu.RESET);
        return;
    }

```

```

System.out.println(startMenu.GREEN +
"Item Code No.: " + startMenu.RESET
+ queue[front][0] + " | " +
startMenu.GREEN + "Item Description: "
+ startMenu.RESET + queue[front][1] +
" | " + startMenu.GREEN + "Quantity: "
+ startMenu.RESET + queue[front][2] +
" | " + startMenu.GREEN + "Unit Price: "
+ startMenu.RESET + queue[front][3] +
" | " + startMenu.GREEN + "Discount: "
+ startMenu.RESET + queue[front][4]);
}

```

```

// Display elements of the queue

```

```

    public void display() {
        if (isEmpty()) {

```

```

System.out.println(startMenu.RED_BOLD + "Queue is empty." +
startMenu.RESET);
        return;
    }

```

```

System.out.print(startMenu.GREEN_BOLD + "Queue elements: " +
startMenu.RESET);
        for (int i = 0; i != size; i++) {

```

```

System.out.print(startMenu.GREEN +
"\nItem Code No.: " +
startMenu.RESET + queue[front + i][0] +
" | " + startMenu.GREEN + "Item
Description: " + startMenu.RESET +
queue[front + i][1] + " | " +

```

```

startMenu.GREEN + "Quantity: " +
startMenu.RESET + queue[front + i][2] +
" | " + startMenu.GREEN + "Unit Price: "
+ startMenu.RESET + queue[front +
i][3] + " | " + startMenu.GREEN +
"Discount: " + startMenu.RESET +
queue[front + i][4]);
    }
    System.out.println();
}
public int size(){
    return size;
}
}

```

HashTable

```

class Data {
    int id; // Key
    Product value; // Value
}

```

```

public class HashTable {
    static int counter = 0;
    static int size;
    Data[] table;
    public HashTable() {
        table = new Data[size];
        for (int i = 0; i < size; i++) {
            table[i] = null;
        }
    }
}

```

```

        public boolean add(int id, String
ITMNO, String QTY, String UPRICE,
String DISC, String ITMDESC) {
            Product product = new
Product(ITMNO, QTY, UPRICE, DISC,
ITMDESC);
            int index = id % size;
            for (int i = 0; i < size; i++) {

```

```

                if (table[index] == null ||
table[index].id == -1) {
                    table[index] = new Data();
                    table[index].id = id;
                    table[index].value = product;
                    return true;
                }
                index = (index + 1) % size; //

```

Rehashing

```

        counter++;
    }

```

```

System.out.println(startMenu.RED_BOLD
+ "Hash table is full!" +
startMenu.RESET);
        return false;
    }

```

```

        public void display() {
            for (int i = 0; i < size; i++) {
                if (table[i] != null && table[i].id !=
-1) {
                    System.out.println("Index " + i
+ ": ID=" + table[i].id + ", Value=" +
table[i].value);
                } else {
                    System.out.println("Index " + i
+ ": Empty");
                }
            }
        }
    }
}

```

```

        public boolean delete(int id) {
            int index = id % size;

            for (int i = 0; i < size; i++) {
                if (table[index] != null &&
table[index].id == id) {
                    table[index].id = -1; // Mark as
deleted
                    table[index].value = 0;

```

```

        return true;
    }
    index = (index + 1) % size; //
Rehashing
    counter++;
}

System.out.println(startMenu.RED_BOLD + "ID not found!" +
startMenu.RESET);
    return false;
}
}

```

Item

```

public abstract class Item {
    public void addItem(){

    }

    public void displayItem(){

    }
    public void deleteItem(){

    }
    public void editDetails(){

    }
}

```

LinkedListQueue

```

public class LinkedListQueue {
    private queueNode front;
    private queueNode rear;
    private int size;
    // Constructor to initialize queue
    public LinkedListQueue() {
        front = null;
        rear = null;
    }
}

```

```

        size = 0;
    }
    // Check if the queue is empty
    public boolean isEmpty() {
        return front == null;
    }

    // Enqueue operation to add an
    element at the rear
    public void enqueue(String ITMNO,
String QTY, String UPRICE, String
DISC, String ITMDESC) {
        Product record = new
Product(ITMNO,QTY,UPRICE,DISC,ITM
DESC);
        queueNode newQueueNode = new
queueNode(record);
        if (rear == null) { // If queue is
empty
            front = rear = newQueueNode;
        } else {
            rear.next = newQueueNode;
            rear = newQueueNode;
        }
        size++;

        System.out.println(startMenu.GREEN +
"Enqueued: " + startMenu.RESET +
record);
    }

    // Dequeue operation to remove an
    element from the front
    public void dequeue() {
        if (isEmpty()) {

        }

        System.out.println(startMenu.RED_BOLD + "Queue is empty! Cannot dequeue."
+ startMenu.RESET);
        }else {
            Product dequeuedValue =
front.data;

```

```

        front = front.next;
        if (front == null) { // If queue
becomes empty
            rear = null;
        }
        size--;

System.out.println(startMenu.GREEN +
"Dequeued: " + startMenu.RESET +
dequeuedValue);
    }
}

// Peek operation to see the front
element
public void peek() {
    if (isEmpty()) {

System.out.println(startMenu.RED_BOLD + "Queue is empty! No front element."
+ startMenu.RESET);
    }else {

System.out.println(startMenu.GREEN +
"Product: " + startMenu.RESET +
front.data);
    }
}

// Display elements of the queue
public void display() {
    if (isEmpty()) {

System.out.println(startMenu.RED_BOLD + "Queue is empty." +
startMenu.RESET);
        return;
    }

System.out.print(startMenu.GREEN +
"Queue elements: " +
startMenu.RESET);

```

```

        queueNode temp = front;
        while (temp != null) {
            System.out.print(temp.data + "
");
            temp = temp.next;
        }
        System.out.println();
    }

// Get List Size
public int size(){
    return size;
}
}

```

Main

```

import java.util.*;

public class Main {
    static Scanner sc = new
Scanner(System.in);
    static String choice;
    static final TwoDimensional twoD =
new TwoDimensional();
    static final OneDimensional oneD =
new OneDimensional();
    static final arrApplication app = new
arrApplication();
    static boolean ifStop;

    static void MainMenu() {
        while(true) {

System.out.println(startMenu.GREEN_BOLD + "\nARRAY & POINTERS: " +
startMenu.RESET);

System.out.println(startMenu.GREEN +
"[1]" + startMenu.RESET + " One
Dimensional Array");

```

```
System.out.println(startMenu.GREEN +
"[2]" + startMenu.RESET + " Two
Dimensional Array");
```

```
System.out.println(startMenu.GREEN +
"[3]" + startMenu.RESET + " Sample
Applications");
```

```
System.out.println(startMenu.GREEN +
"[0]" + startMenu.RESET + " Return to
Main Menu");
```

```
choice = sc.next();
switch (choice) {
    case "1":
        oneD.mainMenu();
        break;
    case "2":
        twoD.mainMenu();
        break;
    case "3":
        app.mainMenu();
        break;
    case "0":
```

```
System.out.println(startMenu.YELLOW_
BOLD + "Returning..." +
startMenu.RESET);
    mainMenu.openMenu();
    break;
    default:
```

```
System.out.println(startMenu.RED_BOL
D + "\nInvalid Choice Selected." +
startMenu.RESET);
    }
    }
    }
}
```

OneDimensional

```
import java.util.*;
public class OneDimensional extends
Item{
```

```
    static Scanner sc = new
Scanner(System.in);
    static ArrayList<String> itmno = new
ArrayList<>();
    static ArrayList<String> itmdesc =
new ArrayList<>();
    static ArrayList<String> qty = new
ArrayList<>();
    static ArrayList<String> uprice = new
ArrayList<>();
    static ArrayList<String> disc = new
ArrayList<>();
    String ITMDESC,ITMNO,QTY,
UPRICE, DISC, choice;
    int ctr = 0;
    static boolean ifStop;
    public void mainMenu(){
        while(!ifStop){
```

```
System.out.println(startMenu.GREEN +
"[1]" + startMenu.RESET + " Add");
```

```
System.out.println(startMenu.GREEN +
"[2]" + startMenu.RESET + " Edit");
```

```
System.out.println(startMenu.GREEN +
"[3]" + startMenu.RESET + " Delete");
```

```
System.out.println(startMenu.GREEN +
"[4]" + startMenu.RESET + " Display
Item");
```

```
System.out.println(startMenu.GREEN +
```

```
"[0]" + startMenu.RESET + " Back to
Array Main Menu");
```

```
choice = sc.next();
```

```
switch(choice) {
```

```
case "1":
```

```
addItem();
```

```
break;
```

```
case "2":
```

```
editDetails();
```

```
break;
```

```
case "3":
```

```
deleteItem();
```

```
break;
```

```
case "4":
```

```
displayItem();
```

```
break;
```

```
case "0":
```

```
System.out.println(startMenu.YELLOW_
BOLD + "Returning..." +
startMenu.RESET);
```

```
ifStop = true;
```

```
break;
```

```
default:
```

```
System.out.println(startMenu.RED_BOL
D + "Wrong Input!" +
startMenu.RESET);
```

```
break;
```

```
}
```

```
}
```

```
}
```

```
public void addItem(){
```

```
ITMNO =
```

```
validateINT.validateInteger(startMenu.G
REEN + "Enter the Item No. : " +
startMenu.RESET);
```

```
System.out.print(startMenu.GREEN +
```

```
"Enter the Description of the Item: " +
startMenu.RESET);
```

```
ITMDESC = sc.next();
```

```
QTY =
```

```
validateINT.validateInteger(startMenu.G
REEN + "Enter the quantity of the Item:
" + startMenu.RESET);
```

```
UPRICE =
```

```
validateDouble.validateDouble(startMen
u.GREEN + "Enter the price of the Item:
" + startMenu.RESET);
```

```
DISC =
```

```
validateDouble.validateDouble(startMen
u.GREEN + "Enter the discount of the
Item: " + startMenu.RESET);
```

```
ctr++;
```

```
itmno.add(ITMNO);
```

```
itmdesc.add(ITMDESC);
```

```
qty.add(QTY);
```

```
uprice.add(UPRICE);
```

```
disc.add(DISC);
```

```
}
```

```
public void displayItem() {
```

```
for(int x = 0; x < ctr; x++){
```

```
System.out.print(startMenu.GREEN +
"Item No: " + startMenu.RESET);
```

```
System.out.println(itmno.get(x));
```

```
System.out.print(startMenu.GREEN +
"Item Description: " +
startMenu.RESET);
```

```
System.out.println(itmdesc.get(x));
```

```
System.out.print(startMenu.GREEN +
"Item Quantity: " + startMenu.RESET);
```

```
System.out.println(qty.get(x));
```

```

System.out.print(startMenu.GREEN +
"Item Price: " + startMenu.RESET);
        System.out.println(uprice.get(x));

System.out.print(startMenu.GREEN +
"Discount: " + startMenu.RESET);
        System.out.println(disc.get(x));

System.out.println("-----");
    }
}
public void editDetails(){
    if(ctr == 0){

System.out.println(startMenu.RED_BOLD + "No item found in the system." +
startMenu.RESET);
    }
    else{
        String ID =
validateINT.validateInteger(startMenu.GREEN + "Enter the ID of the Item: " +
startMenu.RESET);
        for(int x = 0; x < ctr; x++){
            if(Integer.parseInt(ID) ==
Integer.parseInt(itmno.get(x))){

System.out.println(startMenu.GREEN +
"[1]" + startMenu.RESET +
"Description");

System.out.println(startMenu.GREEN +
"[2]" + startMenu.RESET + "Quantity");

System.out.println(startMenu.GREEN +
"[3]" + startMenu.RESET + "Price");

```

```

System.out.println(startMenu.GREEN +
"[4]" + startMenu.RESET + "Discount");
        choice = sc.next();
        switch(choice){
            case "1":

System.out.print(startMenu.GREEN +
"Enter the new Description: " +
startMenu.RESET);
                String Desc =
sc.next();
                itmdesc.set(x, Desc);
                break;
            case "2":
                String Quantity =
validateINT.validateInteger(startMenu.GREEN + "Enter the new Quantity: " +
startMenu.RESET);
                qty.set(x, Quantity);
                break;
            case "3":
                String Price =
validateDouble.validateDouble(startMenu.GREEN + "Enter the new Price: " +
startMenu.RESET);
                uprice.set(x, Price);
                break;
            case "4":
                String Disc =
validateDouble.validateDouble(startMenu.GREEN + "Enter the new Discount: " +
startMenu.RESET);
                disc.set(x, Disc);
                break;
            default:
                break;
        }
    }
}

```

```

    }
}
public void deleteItem(){
    String ID =
validateINT.validateInteger(startMenu.GREEN + "Enter the ID of the Item: " +
startMenu.RESET);
    for(int x = 0; x < ctr; x++){
        if(Integer.parseInt(ID) ==
Integer.parseInt(itmno.get(x))){
            itmno.remove(x);
            itmdesc.remove(x);
            qty.remove(x);
            uprice.remove(x);
            disc.remove(x);
            ctr--;
        }
    }
}
}
}
}
}

```

Product

```

public class Product {
    String ITMDESC, UPRICE, DISC,
ITMNO, QTY;

    Product(String ITMNO, String QTY,
String UPRICE, String DISC, String
ITMDESC) {
        this.ITMNO = ITMNO;
        this.QTY = QTY;
        this.UPRICE = UPRICE;
        this.DISC = DISC;
        this.ITMDESC = ITMDESC;
    }

    @Override
    public String toString(){

```

```

        return startMenu.GREEN + "\nItem
Code No.: " + startMenu.RESET +
ITMNO +
        " | " + startMenu.GREEN +
"Item Description: " + startMenu.RESET
+ ITMDESC +
        " | " + startMenu.GREEN +
"Quantity: " + startMenu.RESET + QTY
+
        " | " + startMenu.GREEN +
"Unit Price: " + startMenu.RESET +
UPRICE +
        " | " + startMenu.GREEN +
"Discount: " + startMenu.RESET +
DISC;
    }
}

```

SinglyProduct

```

import java.util.Scanner;

```

```

//Singly class to

```

```

class SinglyNode {
    SinglyProduct product;
    SinglyNode next;

    SinglyNode(SinglyProduct product) {
        this.product = product;
        this.next = null;
    }
}

```

```

class singlyLinkedList {
    SinglyNode head;

```

```

        public void
newSinglyProduct(SinglyProduct
product) {
            SinglyNode newNode = new
SinglyNode(product);

```

```

        if (head == null) {
            head = newNode;
        } else {
            SinglyNode temp = head;
            while (temp.next != null) {
                temp = temp.next;
            }
            temp.next = newNode;
        }
    }

    public void display() {
        if (head == null) {

System.out.println(startMenu.RED_BOLD + "List is empty" + startMenu.RESET);
            return;
        } else {
            SinglyNode temp = head;
            while (temp != null) {

System.out.println(temp.product);
                temp = temp.next;
            }
        }
    }

    public void delete(int ITMNO) {
        if (head == null) {

System.out.println(startMenu.RED_BOLD + "List is empty" + startMenu.RESET);
            return;
        }

        if (head.product.ITMNO == ITMNO)
    {
        head = head.next;
        return;
    }
}

```

```

        SinglyNode temp = head;
        while (temp.next != null &&
temp.next.product.ITMNO != ITMNO) {
            temp = temp.next;
        }

        if (temp.next == null) {

System.out.println(startMenu.RED_BOLD + "Product not found" +
startMenu.RESET);
        } else {
            temp.next = temp.next.next;
        }
    }
}

public class SinglyProduct {
    int ITMNO;
    String ITMDESC;
    int QTY;
    double UPRICE;
    double DISC;

    public SinglyProduct(int ITMNO,
String ITMDESC, int QTY, double
UPRICE, double DISC) {
        this.ITMNO = ITMNO;
        this.ITMDESC = ITMDESC;
        this.QTY = QTY;
        this.UPRICE = UPRICE;
        this.DISC = DISC;
    }

    @Override
    public String toString() {
        return startMenu.GREEN + "Item
Code No.: " + startMenu.RESET +
ITMNO +

```

```

        ", " + startMenu.GREEN +
        "Item Description: " + startMenu.RESET
        + ITMDESC +
        ", " + startMenu.GREEN +
        "Quantity: " + startMenu.RESET + QTY
        +
        ", " + startMenu.GREEN +
        "Unit Price: " + startMenu.RESET +
        UPRICE +
        ", " + startMenu.GREEN +
        "Discount: " + startMenu.RESET +
        DISC;
    }

```

```

        public static void
        singlyLinkedListMenu() {
            singlyLinkedList addProduct = new
            singlyLinkedList();
            Scanner input = new
            Scanner(System.in);
            System.out.println();

```

```

        System.out.println(startMenu.GREEN_B
        OLD + "Singly Linked List");

```

```

        boolean flag = true;

```

```

        while(flag==true){
            System.out.println("\nInventory
            Management System" +
            startMenu.RESET); //di ko alam name

```

```

        System.out.println(startMenu.GREEN +
        "1." + startMenu.RESET + " Add
        Product");

```

```

        System.out.println(startMenu.GREEN +
        "2." + startMenu.RESET + " Display
        Products");

```

```

        System.out.println(startMenu.GREEN +
        "3." + startMenu.RESET + " Remove a
        Product");

```

```

        System.out.println(startMenu.GREEN +
        "0." + startMenu.RESET + " Back to
        Linked List Menu");

```

```

        System.out.print(startMenu.GREEN +
        "Choose an action: " +
        startMenu.RESET);
        String choice = input.next();

```

```

        switch (choice) {
            case "1":

```

```

                int ITMNO =
                Integer.parseInt(validateINT.validateInte
                ger(startMenu.GREEN + "Enter Item
                Code No.: " + startMenu.RESET));

```

```

                System.out.print(startMenu.GREEN +
                "Enter Item Description: " +
                startMenu.RESET);

```

```

                String ITMDESC =
                input.next();

```

```

                int QTY =
                Integer.parseInt(validateINT.validateInte
                ger(startMenu.GREEN + "Enter
                Quantity: " + startMenu.RESET));

```

```

                double UPRICE =
                Double.parseDouble(validateDouble.vali
                dateDouble(startMenu.GREEN + "Enter
                Unit Price: " + startMenu.RESET));

```

```

                double DISC =
                Double.parseDouble(validateDouble.vali
                dateDouble(startMenu.GREEN + "Enter
                Discount: " + startMenu.RESET));

```

```

                addProduct.newSinglyProduct(new

```

```

SinglyProduct(ITMNO, ITMDESC, QTY,
UPRICE, DISC));
    break;

    case "2":
        addProduct.display();
        System.out.println("");
        break;
    case "3":
        addProduct.display();
        System.out.println("");
        ITMNO =
Integer.parseInt(validateINT.validateInteger(startMenu.GREEN + "Enter Product
ID Code to be removed: " +
startMenu.RESET));
        addProduct.delete(ITMNO);

        break;

    case "0":

openLinkedListMenu.openMenu();
        break;
    default:

System.out.println(startMenu.RED_BOLD + "Invalid Input" + startMenu.RESET);
        break;
    }
}
}
}

```

StackArray

```

public class StackArray {
    private String[][] stack;
    private int top;
    private int maxSize;
    // Constructor to initialize stack

```

```

    public StackArray(int size) {
        maxSize = size;
        stack = new String[maxSize][4];
        top = -1;
    }
    public boolean isFull() {
        return top == maxSize - 1;
    }
    // Push method to add element to the
    stack
    public void push(String ITMNO, String
QTY, String UPRICE, String DISC,
String ITMDESC) {
        Product product = new
Product(ITMNO, QTY, UPRICE, DISC,
ITMDESC);
        if (top == maxSize - 1) {

System.out.println(startMenu.RED_BOLD + "Stack Overflow! Cannot add: " +
startMenu.RESET + product);
            return;
        }
        stack[++top] = new String[]{ITMNO,
ITMDESC, QTY, UPRICE, DISC};

System.out.println(startMenu.GREEN +
"Pushed " + startMenu.RESET +
product);
    }
    // Pop method to remove top element
    public void pop() {
        if (isEmpty()) {

System.out.println(startMenu.RED_BOLD + "Stack Underflow!" +
startMenu.RESET);
            return;
        }
        String[] value = stack[top--];

```

```

System.out.println(startMenu.GREEN +
"Popped Item Code No.: " +
startMenu.RESET + value[0] +
" | " + startMenu.GREEN + "
Item Description: " + startMenu.RESET
+ value[1] +
" | " + startMenu.GREEN +
"Quantity: " + startMenu.RESET +
value[2] +
" | " + startMenu.GREEN +
"Unit Price: " + startMenu.RESET +
value[3] +
" | " + startMenu.GREEN +
"Discount: " + startMenu.RESET +
value[4]);
}
// Peek method to get the top element
without removing it
public void peek() {
    if (isEmpty()) {

System.out.println(startMenu.RED_BOLD
D + "Stack is empty." +
startMenu.RESET);
        return;
    }

```

```

System.out.println(startMenu.GREEN +
"Item Code No.: " + startMenu.RESET +
stack[top][0] +
" | " + startMenu.GREEN +
"Item Description: " + startMenu.RESET
+ stack[top][1] +
" | " + startMenu.GREEN +
"Quantity: " + startMenu.RESET +
stack[top][2] +
" | " + startMenu.GREEN +
"Unit Price: " + startMenu.RESET +
stack[top][3] +

```

```

" | " + startMenu.GREEN +
"Discount: " + startMenu.RESET +
stack[top][4]);
}
// isEmpty method to check if the
stack is empty
public boolean isEmpty() {
    return top == -1;
}
// size method to get the current
number of elements in the stack
public int size() {
    return top + 1;
}
// Display elements in the stack
public void display() {
    if (isEmpty()) {

System.out.println(startMenu.RED_BOLD
D + "Stack is empty." +
startMenu.RESET);
        return;
    }
    System.out.print("Stack
elements:");
    for (int i = 0; i <= top; i++) { // Iterate
through rows

System.out.print(startMenu.GREEN +
"\nItem Code No.: " + startMenu.RESET
+ stack[i][0] +
" | " + startMenu.GREEN +
"Item Description: " + startMenu.RESET
+ stack[i][1] +
" | " + startMenu.GREEN +
"Quantity: " + startMenu.RESET +
stack[i][2] +
" | " + startMenu.GREEN +
"Unit Price: " + startMenu.RESET +
stack[i][3] +

```

```

        " | " + startMenu.GREEN +
"Discount: " + startMenu.RESET +
stack[i][4]);
    }
}
}

```

StackLinkedList

```

public class StackLinkedList {
    private StackNode top;
    private int size;

    // Constructor to initialize stack
    public StackLinkedList() {
        top = null;
        size = 0;
    }

    // Push method to add element to the
    stack
    public void push(String ITMNO, String
    QTY, String UPRICE, String DISC,
    String ITMDESC) {
        Product newProduct = new
    Product(ITMNO, QTY, UPRICE, DISC,
    ITMDESC);
        StackNode newStackNode = new
    StackNode(newProduct);
        newStackNode.next = top;
        top = newStackNode;
        size++;

    System.out.println(startMenu.GREEN +
    "Pushed " + startMenu.RESET +
    newProduct);
    }

    // Pop method to remove top element
    public void pop() {
        if (isEmpty()) {

```

```

    System.out.println(startMenu.RED_BOLD
    + "Stack empty. Cannot Pop." +
    startMenu.RESET);
        }else {

```

```

    System.out.println(startMenu.GREEN +
    "Popped Product: " + startMenu.RESET
    + top.data);
        top = top.next;
        size--;
    }
}

```

```

    // Peek method to get the top element
    without removing it
    public void peek() {
        if (isEmpty()) {

```

```

    System.out.println(startMenu.RED_BOLD
    + "Stack is empty." +
    startMenu.RESET);
        }else{

```

```

    System.out.println(startMenu.GREEN +
    "Most Recent Product: " +
    startMenu.RESET + top.data);
    }
}

```

```

    // isEmpty method to check if the
    stack is empty
    public boolean isEmpty() {
        return top == null;
    }

```

```

    // size method to get the current
    number of elements in the stack
    public int size() {
        return size;
    }

```

```

    }

    // Display elements in the stack
    public void display() {
        if (isEmpty()) {

            System.out.println(startMenu.RED_BOLD + "Stack is empty." + startMenu.RESET);
            return;
        }

        System.out.print(startMenu.GREEN + "Stack elements: " + startMenu.RESET);
        StackNode temp = top;
        while (temp != null) {
            System.out.print(temp.data + "
");
            temp = temp.next;
        }
        System.out.println();
    }
}

```

StackNode

```

public class StackNode {
    Product data;
    StackNode next;
    StackNode(Product data) {
        this.data = data;
        next = null;
    }
}

```

TwoDimensional

```

import java.util.*;

public class TwoDimensional extends Item{

```

```

    Scanner sc = new
Scanner(System.in);
    static ArrayList<ArrayList<String>>
getInfo = new ArrayList<>();
    static int ctr = 0;
    String ITMDESC, ITMNO,QTY,
UPRICE, DISC, choice;
    static boolean ifStop;
    public void mainMenu(){
        while(!ifStop){

            System.out.println(startMenu.GREEN_BOLD + "TWO DIMENSIONAL ARRAY" + startMenu.RESET);

            System.out.println(startMenu.GREEN + "[1]" + startMenu.RESET + " Add");

            System.out.println(startMenu.GREEN + "[2]" + startMenu.RESET + " Edit");

            System.out.println(startMenu.GREEN + "[3]" + startMenu.RESET + " Delete");

            System.out.println(startMenu.GREEN + "[4]" + startMenu.RESET + " Display Item");

            System.out.println(startMenu.GREEN + "[0]" + startMenu.RESET + " Back to Array Main Menu");
            choice = sc.next();
            switch(choice) {
                case "1":
                    addItem();
                    break;
                case "2":
                    editDetails();
                    break;
                case "3":

```

```

        deleteItem();
        break;
    case "4":
        displayItem();
        break;
    case "0":
        ifStop = true;
        break;
    default:

System.out.println(startMenu.RED_BOLD + "Wrong Input!" + startMenu.RESET);
        break;
    }
}

public void addItem(){
    ITMNO =
validateINT.validateInteger(startMenu.GREEN + "Enter the Item No. : " +
startMenu.RESET);

System.out.print(startMenu.GREEN +
"Enter the Description of the Item: " +
startMenu.RESET);
    ITMDESC = sc.next();

    QTY =
validateINT.validateInteger(startMenu.GREEN + "Enter the quantity of the Item:
" + startMenu.RESET);

    UPRICE =
validateDouble.validateDouble(startMenu.GREEN + "Enter the price of the Item:
" + startMenu.RESET);

    DISC =
validateDouble.validateDouble(startMenu.GREEN + "Enter the discount of the
Item: " + startMenu.RESET);

```

```

        ctr++;

        getInfo.add(new
ArrayList<String>(Arrays.asList(ITMNO,
ITMDESC, QTY, UPRICE, DISC)));
    }

    public void displayItem(){

System.out.println(startMenu.GREEN_BOLD + "DISPLAY ITEMS" +
startMenu.RESET);

System.out.println(startMenu.GREEN +
"[1]" + startMenu.RESET + " Display
Item in Array");

System.out.println(startMenu.GREEN +
"[2]" + startMenu.RESET + " Display
Item in Organized");

System.out.println(startMenu.GREEN +
"[3]" + startMenu.RESET + " Back to
2-D Main Menu");
        choice =sc.next();

        switch(choice){
            case "1":
                for(int x = 0; x < ctr; x++){
                    System.out.print((x+1) + ".
");

System.out.println(getInfo.get(x));
                }
                break;
            case "2":

System.out.println("-----");
                for(int x = 0; x < ctr; x++){

```

```

System.out.print(startMenu.GREEN +
"Item No: " + startMenu.RESET );

System.out.println(getInfo.get(x).get(0));

System.out.print(startMenu.GREEN +
"Item Description: " + startMenu.RESET
);

System.out.println(getInfo.get(x).get(1));

System.out.print(startMenu.GREEN +
"Item Quantity: " + startMenu.RESET );

System.out.println(getInfo.get(x).get(2));

System.out.print(startMenu.GREEN +
"Item Price: " + startMenu.RESET );

System.out.println(getInfo.get(x).get(3));

System.out.print(startMenu.GREEN +
"Discount: " + startMenu.RESET );

System.out.println(getInfo.get(x).get(4));

System.out.println("-=====
-");
    }
    break;
case "3":
    break;
default:
    break;
}
}

public void editDetails(){

```

```

        if(ctr == 0){

System.out.println(startMenu.RED_BOLD + "No item found in the system." +
startMenu.RESET );
        }
        else{

System.out.print(startMenu.GREEN +
"Enter the ID of the Item: " +
startMenu.RESET);
            String ID = sc.next();
            for(int x = 0; x < ctr; x++){
                if(Integer.parseInt(ID) ==
Integer.parseInt(getInfo.get(x).get(0))){

System.out.println(startMenu.GREEN +
"[1]" + startMenu.RESET +
"Description");

System.out.println(startMenu.GREEN +
"[2]" + startMenu.RESET + "Quantity");

System.out.println(startMenu.GREEN +
"[3]" + startMenu.RESET + "Price");

System.out.println(startMenu.GREEN +
"[4]" + startMenu.RESET + "Discount");
                    choice = sc.next();
                    switch(choice){
                        case "1":

System.out.print(startMenu.GREEN +
"Enter the new Description: " +
startMenu.RESET);

                                String Desc =
sc.next();

                                    getInfo.get(x).set(1,
Desc);

                                            break;

```

```

                case "2":
                    String Quantity =
validateINT.validateInteger(startMenu.G
REEN + "Enter the new Quantity: " +
startMenu.RESET);
                    getInfo.get(x).set(2,
Quantity);
                    break;
                case "3":
                    String Price =
validateDouble.validateDouble(startMen
u.GREEN + "Enter the new Price: " +
startMenu.RESET);
                    getInfo.get(x).set(3,
Price);
                    break;
                case "4":
                    String Disc =
validateDouble.validateDouble(startMen
u.GREEN + "Enter the new Discount: " +
startMenu.RESET);
                    getInfo.get(x).set(4,
Disc);
                    break;
                default:
                    break;
            }
        }
    }
}

public void deleteItem(){
    String ID =
validateINT.validateInteger(startMenu.G
REEN + "Enter the ID of the Item: " +
startMenu.RESET);
    for(int x = 0; x < ctr; x++){
        if(Integer.parseInt(ID) ==
Integer.parseInt(getInfo.get(x).get(0))){

```

```

        getInfo.remove(x);
        ctr--;
    }
}
}
}
}

```

arrApplication

```

import java.util.Scanner;

public class arrApplication {
    static Scanner sc = new
Scanner(System.in);
    static connectFour_Application
conFour = new
connectFour_Application();
    static tictactoe_Application ttToe =
new tictactoe_Application();
    Main main = new Main();
    String user;
    public void mainMenu(){

System.out.println(startMenu.GREEN +
"[1]" + startMenu.RESET + "Tic Tac
Toe");

System.out.println(startMenu.GREEN +
"[2]" + startMenu.RESET + "Connect
Four");

System.out.println(startMenu.GREEN +
"[0]" + startMenu.RESET + "Back to
Main Menu");
        user = sc.next();
        switch(user){
            case "1":
                ttToe.ticTacToe();

System.out.println(startMenu.YELLOW_

```

```

        BOLD +"Going back to the menu..." +
        startMenu.RESET);
            break;
        case "2":
            conFour.connectFour();

        System.out.println(startMenu.YELLOW_
        BOLD + "Going back to the menu..." +
        startMenu.RESET);
            break;
        case "0":

        System.out.println(startMenu.YELLOW_
        BOLD + "Returning..." +
        startMenu.RESET);
            main.MainMenu();
            break;
        default:

        System.out.println(startMenu.RED_BOL
        D + "Wrong Input!" +
        startMenu.RESET);
            break;
        }
    }
}

```

connectFour_Application

```

import java.util.Scanner;

public class connectFour_Application {

    public static String arr[][] = { { "1",
    "2", "3", "4", "5" },
        { "O", "O", "O", "O", "O" },
        { "O", "O", "O", "O", "O" },
        { "O", "O", "O", "O", "O" },
        { "O", "O", "O", "O", "O" },
        { "O", "O", "O", "O", "O" } };

```

```

        public static String arr2[][] = { { "1",
        "2", "3", "4", "5" }, { "O", "O", "O", "O",
        "O"},
            { "O", "O", "O", "O", "O"},
            { "O", "O", "O", "O", "O"},
            { "O", "O", "O", "O", "O"},
            { "O", "O", "O", "O", "O" } };

        public static int ctr = 0, col, win = 0,
        flag = 0, draw = 0, flag_move = 0, move;
        public static String P1 = "P1";
        static Scanner sc = new
        Scanner(System.in);

        static void resetBoard(){
            for (int x = 0; x < 6; x++) {
                for (int y = 0; y < 5; y++) {
                    arr[x][y] = arr2[x][y];
                }
            }
        }

        public static void showBoard() {
            for (int x = 0; x < 6; x++) {
                for (int y = 0; y < 5; y++) {
                    System.out.print(arr[x][y] + "
                ");
            }
            System.out.println();
        }
    }
}

```

```

        public static void MakeMove() {
            if (P1.equals("P1")) {
                move = sc.nextInt();
                flag_move = 0;
                ctr++;
            } else if (P1.equals("P2")) {
                move = sc.nextInt();
                flag_move = 0;
            }
        }
    }
}

```

```

        ctr++;
    }
    switch (move) {
        case 1:
            col = 0;
            break;
        case 2:
            col = 1;
            break;
        case 3:
            col = 2;
            break;
        case 4:
            col = 3;
            break;
        case 5:
            col = 4;
            break;
    }
}

public static void PlaceMove() {
    for (int x = 5; x > 0 && flag_move
== 0; x--) {
        // checks if current position is
not filled with "Y" and "R"
        if (!(arr[x][col].equals("Y")) &&
!(arr[x][col].equals("R")) && P1 == "P1")
        {
            arr[x][col] = "R";
            P1 = "P2";
            flag_move = 1;
        } else if
(! (arr[x][col].equals("R")) &&
! (arr[x][col].equals("Y")) && P1 == "P2")
        {
            arr[x][col] = "Y";
            P1 = "P1";
            flag_move = 1;
        }
    }
}

```

```

    }
}

public static int checkWinner() {
    flag = 0;
    if (ctr > 19) {
        flag = 1;
    }
    // check for straight patterns
    for (int x = 5; x > 0; x--) {
        if ((arr[x][0] == arr[x][1] &&
arr[x][1] == arr[x][2] && arr[x][2] ==
arr[x][3])
            || (arr[x][1] == arr[x][2] &&
arr[x][2] == arr[x][3] && arr[x][3] ==
arr[x][4])) {
            win = 1;
            flag = 1;
        }
        for (int y = 0; y < 5; y++) {
            if ((arr[1][y] == arr[2][y] &&
arr[2][y] == arr[3][y] && arr[3][y] ==
arr[4][y])
                || arr[2][y] == arr[3][y]
&& arr[3][y] == arr[4][y] && arr[4][y] ==
arr[5][y])) {
                win = 1;
                flag = 1;
            }
        }
    }
    // check for diagonal patterns
    if ((arr[1][0] == arr[2][1] &&
arr[2][1] == arr[3][2] && arr[3][2] ==
arr[4][3]) ||
        (arr[2][1] == arr[3][2] &&
arr[3][2] == arr[4][3] && arr[4][3] ==
arr[5][4])) {
        win = 1;
        flag = 1;
    }
}

```

```

        } else if ((arr[2][0] == arr[3][1] &&
arr[3][1] == arr[4][2] && arr[4][2] ==
arr[5][3])
                || (arr[1][1] == arr[2][2] &&
arr[2][2] == arr[3][3] && arr[3][3] ==
arr[4][4])) {
            win = 1;
            flag = 1;
        } else if ((arr[1][4] == arr[2][3] &&
arr[2][3] == arr[3][2] && arr[3][2] ==
arr[4][1]) ||
                (arr[2][3] == arr[3][2] &&
arr[3][2] == arr[4][1] && arr[4][1] ==
arr[5][0])) {
            win = 1;
            flag = 1;
        } else if ((arr[2][4] == arr[3][3] &&
arr[3][3] == arr[4][2] && arr[4][2] ==
arr[5][1])
                || (arr[4][0] == arr[3][1] &&
arr[3][1] == arr[2][2] && arr[2][2] ==
arr[1][3])) {
            win = 1;
            flag = 1;
        }
        return flag;
    }

    public static void connectFour() {
        flag = 0;
        win = 0;
        System.out.println("WELCOME
TO CONNECT FOUR");
        System.out.println("PICK 1 2 3 4
5!!, NEEDS 2 PLAYERS");
        System.out.println("GOOO!!!");
        while (flag == 0) {

System.out.println("-----");
            showBoard();

```

```

            System.out.println("-----");
            if (P1 == "P1") {
                System.out.println("TURN
PLAYER = " + P1 + " = R");
            } else {
                System.out.println("TURN
PLAYER = " + P1 + " = Y");
            }
            MakeMove();
            PlaceMove();
            checkWinner();
        }
        showBoard();
        if (P1 == "P2" && win == 1) {

System.out.println("-----");
            System.out.println("P1
WINS");
        } else if (P1 == "P1" && win ==
1) {

System.out.println("-----");
            System.out.println("P2
WINS");
        } else {

System.out.println("-----");
            System.out.println("ITS A
DRAW");
        }
        resetBoard();
    }
}

```

DoublyProductData

```
import java.util.Scanner;
```

```
class DoublyNode {
```

```

doublyProductData product;
DoublyNode next;
DoublyNode prev;

    DoublyNode(doublyProductData
product) {
    this.product = product;
    this.next = null;
    this.prev = null;
}
}

```

```

class doublyLinkedList {
    DoublyNode head;
    DoublyNode tail;

                                public void
newProduct(doublyProductData
product) {
    DoublyNode newNode = new
DoublyNode(product);
    if (head == null) {
        head = tail = newNode;
    } else {
        tail.next = newNode;
        newNode.prev = tail;
        tail = newNode;
    }
}
}

```

```

    public void display() {
        if (head == null) {

System.out.println(startMenu.RED_BOL
D + "List is empty" + startMenu.RESET);
        } else {
            DoublyNode temp = head;
            while (temp != null) {

System.out.println(temp.product);

```

```

                temp = temp.next;
            }
        }
    }

    public void displayReverse() {
        if (tail == null) {

System.out.println(startMenu.RED_BOL
D + "List is empty" + startMenu.RESET);
            return;
        }

        DoublyNode temp = tail;
        while (temp != null) {

System.out.println(temp.product);
            temp = temp.prev;
        }
    }

    public void delete(int ITMNO) {
        if (head == null) {

System.out.println(startMenu.RED_BOL
D + "List is empty" + startMenu.RESET);
            return;
        }

        if (head.product.ITMNO == ITMNO)
        {
            if (head == tail) {
                head = tail = null;
            } else {
                head = head.next;
                head.prev = null;
            }
            return;
        }
    }
}

```

```

        DoublyNode temp = head;
        while (temp != null &&
temp.product.ITMNO != ITMNO) {
            temp = temp.next;
        }

        if (temp == null) {

System.out.println(startMenu.RED_BOLD + "Product not found !" +
startMenu.RESET);
        } else {
            if (temp == tail) {
                tail = tail.prev;
                tail.next = null;
            } else {
                temp.prev.next = temp.next;
                temp.next.prev = temp.prev;
            }
        }
    }
}

```

```

public class doublyProductData {
    int ITMNO;
    String ITMDESC;
    int QTY;
    double UPRICE;
    double DISC;

    public doublyProductData(int ITMNO,
String ITMDESC, int QTY, double
UPRICE, double DISC) {
        this.ITMNO = ITMNO;
        this.ITMDESC = ITMDESC;
        this.QTY = QTY;
        this.UPRICE = UPRICE;
        this.DISC = DISC;
    }
}

```

```

@Override
public String toString() {
    return startMenu.GREEN + "Item
Code No.: " + startMenu.RESET +
ITMNO + ", " + startMenu.GREEN +
"Item Description: " + startMenu.RESET
+ ITMDESC + ", " + startMenu.GREEN
+ "Quantity: " + startMenu.RESET +
QTY +
        ", " + startMenu.GREEN +
"Unit Price: " + startMenu.RESET +
UPRICE + ", " + startMenu.GREEN +
"Discount: " + startMenu.RESET +
DISC;
}

```

```

        public static void
doublyLinkedListMenu() {
            doublyLinkedList productDataLists
= new doublyLinkedList();
            Scanner input =new
Scanner(System.in);
            System.out.println();

```

```

System.out.println(startMenu.GREEN_BOLD + "Doubly Linked List Menu");
        boolean flag =true;

```

```

        while(flag==true){
            System.out.println("\nInventory
Management      System"      +
startMenu.RESET); //di ko alam name

```

```

System.out.println(startMenu.GREEN +
"1." + startMenu.RESET + " Add
Product");

```

```

System.out.println(startMenu.GREEN +
"2." + startMenu.RESET + " Display
Products");

```

```
System.out.println(startMenu.GREEN +
"3." + startMenu.RESET + " Display
Products in Reversed");
```

```
System.out.println(startMenu.GREEN +
"4." + startMenu.RESET + " Remove a
Product");
```

```
System.out.println(startMenu.GREEN +
"0." + startMenu.RESET + " Back to
Linked List Menu");
```

```
System.out.print(startMenu.GREEN +
"Choose an action: " +
startMenu.RESET);
```

```
String choice = input.next();
System.out.println();
```

```
switch (choice) {
case "1":
```

```
int ITMNO =
Integer.parseInt(validateINT.validateInteger(startMenu.GREEN + "Enter Item
Code No.: " + startMenu.RESET));
```

```
System.out.print(startMenu.GREEN +
"Enter Item Description: " +
startMenu.RESET);
```

```
String ITMDESC =
input.next();
```

```
int QTY =
Integer.parseInt(validateINT.validateInteger(startMenu.GREEN + "Enter
Quantity: " + startMenu.RESET));
```

```
double UPRICE =
Double.parseDouble(validateDouble.validateDouble(startMenu.GREEN + "Enter
Unit Price: " + startMenu.RESET));
```

```
double DISC =
Double.parseDouble(validateDouble.validateDouble(startMenu.GREEN + "Enter
Discount: " + startMenu.RESET));
```

```
productDataLists.newProduct(new
doublyProductData(ITMNO, ITMDESC,
QTY, UPRICE, DISC));
```

```
break;
```

```
case "2":
productDataLists.display();
System.out.println("");
break;
```

```
case "3":
```

```
productDataLists.displayReverse();
System.out.println("");
```

```
break;
```

```
case "4":
```

```
productDataLists.display();
```

```
ITMNO =
Integer.parseInt(validateINT.validateInteger("\n"+ startMenu.GREEN + "Enter
Product ID Code to remove from the
List: " + startMenu.RESET));
```

```
productDataLists.delete(ITMNO);
break;
```

```
case "0":
```

```
System.out.println(startMenu.YELLOW_
BOLD + "Returning..." +
startMenu.RESET);
```

```
openLinkedListMenu.openMenu();
break;
```

```

        default:

System.out.println(startMenu.RED_BOLD + "Input not available!" + startMenu.RESET);
    }
}
}
}

```

mainMenu

```

import java.util.Scanner;

public class mainMenu {
    public static void openMenu(){
        Scanner sc = new Scanner(System.in);
        String choice;
        boolean loop = true;
        while (loop){

System.out.println(startMenu.GREEN_BOLD + "\nMAIN MENU " + startMenu.RESET );

System.out.println("-----
-----");

System.out.print(startMenu.GREEN + "[1]" + startMenu.RESET + "Array & Pointers\n" +
        startMenu.GREEN + "[2]" + startMenu.RESET + "Linked Lists\n" +
        startMenu.GREEN + "[3]" + startMenu.RESET + "Stack Data Structure\n" +
        startMenu.GREEN + "[4]" + startMenu.RESET + "Queue Data Structure\n" +

```

```

        startMenu.GREEN + "[5]" + startMenu.RESET + "Search & Sorting Algorithms\n" +
        startMenu.GREEN + "[6]" + startMenu.RESET + "Hash Tables\n" +
        startMenu.GREEN + "[7]" + startMenu.RESET + "Group Members\n" +
        startMenu.GREEN + "[0]" + startMenu.RESET + "Exit\n" +
        startMenu.GREEN + "Enter your Choice -> " + startMenu.RESET);
        choice = sc.next();
        switch (choice){
            case "1":
                Main.MainMenu();
                break;
            case "2":
                openLinkedListMenu.openMenu();
                break;
            case "3":
                openStackMenu.openMenu();
                break;
            case "4":
                openQueueMenu.openMenu();
                break;
            case "5":
                openSearchAndSortMenu.openMenu();
                break;
            case "6":
                String ans = validateINT.validateInteger(
                    startMenu.GREEN + "Size: " + startMenu.RESET);
                HashTable.size = Integer.parseInt(ans);

```

```

openHashTableMenu.openMenu();
    break;
    case "7":

openGroupMembersMenu.openMenu();
    break;
    case "0":
        System.out.println("\nThank
you for using the System.\n"+
startMenu.RED_BOLD + "Exiting...\n" +
startMenu.RESET);
        System.exit(0);
    default:

System.out.println(startMenu.RED_BOLD
+ "\nInvalid Choice Selected." +
startMenu.RESET);
    }
}
}
}
}

```

openGroupMembersMenu

```

import java.util.*;

public class openGroupMembersMenu {
    public static void openMenu(){
        Scanner sc = new
Scanner(System.in);
        String choice;
        boolean loop = true;
        while (loop){

System.out.println(startMenu.GREEN_BOLD
+ "\nGROUP MEMBERS" +
startMenu.RESET);

System.out.println("-=====
=====");

```

```

        System.out.print(
            startMenu.GREEN + "[1]
Group Leader:" + startMenu.RESET + "
Oleriana, Mico Laurenz D.\n" +
            startMenu.GREEN +
"[2] Member 1:" + startMenu.RESET + "
Amojilar, Rogell B.\n" +
            startMenu.GREEN +
"[3] Member 2:" + startMenu.RESET + "
Dela Cruz, Maria Kylee A.\n" +
            startMenu.GREEN +
"[4] Member 3:" + startMenu.RESET + "
Gonzalez, Rheanna V.\n" +
            startMenu.GREEN +
"[5] Member 4:" + startMenu.RESET + "
Manzano, Elohim Lee L.\n" +
            startMenu.GREEN +
"[6] Member 5:" + startMenu.RESET + "
San Pedro, Jerome M.\n" +
            startMenu.GREEN +
"[7] Member 6:" + startMenu.RESET + "
Villanueva, Mikaela Marie A.\n" +
            startMenu.GREEN +
"[0]" + startMenu.RESET + " Back to
Main Menu\n" +
            startMenu.GREEN +
"Enter your Choice -> " +
startMenu.RESET);
        choice = sc.next();
        switch (choice){
            case "1":

System.out.println(startMenu.GREEN +
"\nName:" + startMenu.RESET + "
Oleriana, Mico Laurenz D.\n" +
            startMenu.GREEN +
"Student No.:" + startMenu.RESET + "
2023100190\n" +
            startMenu.GREEN +
"Age:" + startMenu.RESET + " 19\n" +

```

```

                startMenu.GREEN +
"Address:" + startMenu.RESET + "
Bocaue,Bulacan");
        break;
        case "2":

```

```

System.out.println(startMenu.GREEN +
"\nName:" + startMenu.RESET + "
Amojilar, Rogell B.\n" +
                startMenu.GREEN +
"Student No.:" + startMenu.RESET + "
2023100279\n" +
                startMenu.GREEN +
"Age:" + startMenu.RESET + " 19\n" +
                startMenu.GREEN +
"Address:" + startMenu.RESET + " Sta
Maria, Bulacan");
        break;
        case "3":

```

```

System.out.println(startMenu.GREEN +
"\nName:" + startMenu.RESET + " Dela
Cruz, Maria Kylee A.\n" +
                startMenu.GREEN +
"Student No.:" + startMenu.RESET + "
2023100377\n" +
                startMenu.GREEN +
"Age:" + startMenu.RESET + " 19\n" +
                startMenu.GREEN +
"Address:" + startMenu.RESET + "
Plaridel,Bulacan");
        break;
        case "4":

```

```

System.out.println(startMenu.GREEN +
"\nName:" + startMenu.RESET + "
Gonzalez, Rheanna V.\n" +
                startMenu.GREEN +
"Student No.:" + startMenu.RESET + "
2023100385\n" +

```

```

                startMenu.GREEN +
"Age:" + startMenu.RESET + " 19\n" +
                startMenu.GREEN +
"Address:" + startMenu.RESET + " Sta.
Maria,Bulacan");
        break;
        case "5":

```

```

System.out.println(startMenu.GREEN +
"\nName:" + startMenu.RESET + "
Manzano, Elohim Lee L.\n" +
                startMenu.GREEN +
"Student No.:" + startMenu.RESET + "
2023100452\n" +
                startMenu.GREEN +
"Age:" + startMenu.RESET + " 18\n" +
                startMenu.GREEN +
"Address:" + startMenu.RESET + "
Apalit, Pampanga");
        break;
        case "6":

```

```

System.out.println(startMenu.GREEN +
"\nName:" + startMenu.RESET + " San
Pedro, Jerome M.\n" +
                startMenu.GREEN +
"Student No.:" + startMenu.RESET + "
2023100179\n" +
                startMenu.GREEN +
"Age:" + startMenu.RESET + " 19\n" +
                startMenu.GREEN +
"Address:" + startMenu.RESET + "
Pulilan, Bulacan");
        break;
        case "7":

```

```

System.out.println(startMenu.GREEN +
"\nName:" + startMenu.RESET + "
Villanueva, Mikaela Marie A.\n" +

```

```

                startMenu.GREEN +
"Student No.:" + startMenu.RESET + "
2023100341\n" +
                startMenu.GREEN +
"Age:" + startMenu.RESET + " 20\n" +
                startMenu.GREEN +
"Address:" + startMenu.RESET + "
Calumpit, Bulacan");
            break;
            case "0":
                mainMenu.openMenu();
            default:

System.out.println(startMenu.RED_BOLD + "\nInvalid Choice Selected." +
startMenu.RESET);
        }
    }
}

```

openHashFunctionMenu

```

import java.util.Scanner;

public class openHashFunctionMenu{

    public static void openMenu(){
        String ITMDESC, choice, ITMNO,
QTY, UPRICE, DISC, ans;
        int id,answer;
        boolean loop = true,check = false;
        Scanner sc = new
Scanner(System.in);
        while (loop){

System.out.println(startMenu.GREEN_BOLD + "\nHash Table " +
startMenu.RESET);

```

```

System.out.println("-----
-----");

```

```

System.out.print(startMenu.GREEN +
"[1]" + startMenu.RESET + " Add
Product\n" +

```

```

                startMenu.GREEN + "[2]" +
startMenu.RESET + " View Product\n" +
                startMenu.GREEN + "[3]" +
startMenu.RESET + " Delete Product\n"
+

```

```

                startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Queue
Data Structure\n" +

```

```

                startMenu.GREEN + "Enter
your Choice -> " + startMenu.RESET);

```

```

                choice = sc.next();

```

```

                switch (choice){

```

```

                    case "1":

```

```

                        check = false;

```

```

                                for (int x = 0; x !=
openHashTableMenu.hashTable.table.le
ngth;x++){

```

```

if(openHashTableMenu.hashTable.table[
x]== null){

```

```

                    check = true;

```

```

                }

```

```

            }

```

```

            if(check) {

```

```

                                id =

```

```

Integer.parseInt(validateINT.validateInte
ger(startMenu.GREEN + "\nEnter Hash
Table ID: " + startMenu.RESET));

```

```

                                ITMNO =

```

```

validateINT.validateInteger(startMenu.G
REEN + "\nEnter Item Code No.: " +
startMenu.RESET);

```

```

System.out.print(startMenu.GREEN +
"Enter Item Description: " +
startMenu.RESET);
        ITMDESC = sc.next();
        QTY =
validateINT.validateInteger(startMenu.G
REEN + "Enter Quantity: " +
startMenu.RESET);
        UPRICE =
validateDouble.validateDouble(startMen
u.GREEN + "Enter Unit Price: " +
startMenu.RESET);
        DISC =
validateDouble.validateDouble(startMen
u.GREEN + "Enter Discount: " +
startMenu.RESET);

openHashTableMenu.hashTable.add(id,I
TMNO,QTY,UPRICE,DISC,ITMDESC);
    }else{

System.out.println(startMenu.RED_BOL
D + "Hash table is full!" +
startMenu.RESET);
    }
    break;
    case "2":

openHashTableMenu.hashTable.display
();
        break;
    case "3":
        check = false;
        for (int x = 0; x !=
openHashTableMenu.hashTable.table.le
ngth;x++){

if(openHashTableMenu.hashTable.table[
x] != null &&

```

```

openHashTableMenu.hashTable.table[x]
.id != -1){
        check = true;
    }
    }
    if(check) {

openHashTableMenu.hashTable.display
();
        answer =
Integer.parseInt(validateINT.validateInte
ger(startMenu.GREEN + "Enter Hash
Table ID to delete: " +
startMenu.RESET));

openHashTableMenu.hashTable.delete(
answer);
    }else{

System.out.println(startMenu.RED_BOL
D + "Hash table is empty!" +
startMenu.RESET);
    }
    break;
    case "0":

openHashTableMenu.openMenu();
    default:

System.out.println(startMenu.RED_BOL
D + "\nInvalid Choice Selected." +
startMenu.RESET);
    }
    }
}

openHashTableMenu
import java.util.Scanner;

```

```

public class openHashTableMenu {
    public static HashTable hashTable =
new HashTable();
    public static void openMenu(){
        Scanner sc = new
Scanner(System.in);
        String choice;
        boolean loop = true;
        while (loop){

System.out.println(startMenu.GREEN_B
OLD + "\nHASH TABLES" +
startMenu.RESET);

System.out.println("-----
-----");

System.out.print(startMenu.GREEN +
"[1]" + startMenu.RESET + " Hash
Function\n" +
        startMenu.GREEN + "[2]" +
startMenu.RESET + " Hash Collision\n"
+
        startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Main
Menu\n" +
        startMenu.GREEN + "Enter
your Choice -> " + startMenu.RESET);
        choice = sc.next();
        switch (choice){
            case "1":

openHashFunctionMenu.openMenu();
            case "2":

System.out.println(startMenu.GREEN +
"Number of Collisions: " +
startMenu.RESET
HashTable.counter);
            break;

```

```

        case "0":
            mainMenu.openMenu();
        default:

System.out.println(startMenu.RED_BOL
D + "\nInvalid Choice Selected." +
startMenu.RESET);
        }
    }
}

```

openLinkedListMenu

```

import java.util.Scanner;

public class openLinkedListMenu {
    public static void openMenu(){
        Scanner sc = new
Scanner(System.in);
        String choice;
        boolean loop = true;
        while (loop){

System.out.println(startMenu.GREEN_B
OLD + "\nLINKED LIST DATA
STRUCTURE" + startMenu.RESET);

System.out.println("-----
-----");

System.out.print(startMenu.GREEN +
"[1]" + startMenu.RESET + " Singly
Linked List\n" +
        startMenu.GREEN + "[2]" +
startMenu.RESET + " Doubly Linked
List\n" +
        startMenu.GREEN + "[3]" +
startMenu.RESET + " Linked List Class
in Java\n" +

```

```

        startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Main
Menu\n" +
        startMenu.GREEN + "Enter
your Choice -> " + startMenu.RESET);
        choice = sc.next();
        switch (choice){
            case "1":

SinglyProduct.singlyLinkedListMenu();
            case "2":

doublyProductData.doublyLinkedListMe
nu();

            case "3":
                productData.openMenu();
            case "0":
                mainMenu.openMenu();
            default:

System.out.println(startMenu.RED_BOL
D + "\nInvalid Choice Selected." +
startMenu.RESET);
        }
    }
}
}

```

openQueueArray

```

import java.util.Scanner;

public class openQueueArray {
    public static void openMenu(){
        String ITMDESC, choice, ITMNO,
QTY, UPRICE, DISC, ans;
        boolean loop = true;
        Scanner sc = new
Scanner(System.in);

```

```

        ans = validateINT.validateInteger(
startMenu.GREEN + "Array Size: " +
startMenu.RESET);
        ArrayQueue list = new
ArrayQueue(Integer.parseInt(ans));
        while (loop){

System.out.println(startMenu.GREEN_B
OLD + "\nQueue Array " +
startMenu.RESET);

System.out.println("-----
-----");

System.out.print(startMenu.GREEN +
"[1]" + startMenu.RESET + " Enqueue
Product\n" +
        startMenu.GREEN + "[2]" +
startMenu.RESET + " Dequeue
Product\n" +
        startMenu.GREEN + "[3]" +
startMenu.RESET + " Peek Product\n" +
        startMenu.GREEN + "[4]" +
startMenu.RESET + " Display
Product/s\n" +
        startMenu.GREEN + "[5]" +
startMenu.RESET + " List Size\n" +
        startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Queue
Data Structure\n" +
        startMenu.GREEN + "Enter
your Choice -> " + startMenu.RESET);
        choice = sc.next();
        switch (choice){
            case "1":
                if(list.isFull()){

System.out.println(startMenu.RED_BOL
D + "Queue is full! Cannot enqueue
more records." + startMenu.RESET);

```

```

        break;
    }else{
        ITMNO =
validateINT.validateInteger(startMenu.GREEN + "\nEnter Item Code No.: " +
startMenu.RESET);

System.out.print(startMenu.GREEN +
"Enter Item Description: " +
startMenu.RESET);
        ITMDESC = sc.next();
        QTY =
validateINT.validateInteger(startMenu.GREEN + "Enter Quantity: " +
startMenu.RESET);

        UPRICE =
validateDouble.validateDouble(startMenu.GREEN + "Enter Unit Price: " +
startMenu.RESET);

        DISC =
validateDouble.validateDouble(startMenu.GREEN + "Enter Discount: " +
startMenu.RESET);

list.enqueue(ITMNO,QTY,UPRICE,DISC,ITMDESC);
        break;
    }
    case "2":
        list.dequeue();
        break;
    case "3":
        list.peek();
        break;
    case "4":
        list.display();
        break;
    case "5":

```

```
System.out.println(startMenu.GREEN +
```

```

"Size: " + startMenu.RESET +
list.size());
        break;
    case "0":

openQueueMenu.openMenu();
        default:

System.out.println(startMenu.RED_BOLD + "\nInvalid Choice Selected." +
startMenu.RESET);
    }
    }
    }
}

```

openQueueInClass

```

import java.util.LinkedList;
import java.util.Queue;
import java.util.Scanner;

public class openQueueInClass {
    public static void openMenu(){
        String ITMDESC, choice, ITMNO,
QTY, UPRICE, DISC;
        boolean loop = true;
        Scanner sc = new
Scanner(System.in);
        Queue<Product> list = new
LinkedList<>();
        while (loop){

System.out.println(startMenu.GREEN_BOLD + "\nQueue In Class " +
startMenu.RESET);

System.out.println("-----
-----");

```

```
System.out.print(startMenu.GREEN +
```

```

"[1]" + startMenu.RESET + " Enqueue
Product\n" +
        startMenu.GREEN + "[2]" +
startMenu.RESET + " Dequeue
Product\n" +
        startMenu.GREEN + "[3]" +
startMenu.RESET + " Peek Product\n" +
        startMenu.GREEN + "[4]" +
startMenu.RESET + " Display
Product/s\n" +
        startMenu.GREEN + "[5]" +
startMenu.RESET + " List Size\n" +
        startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Queue
Data Structure\n" +
        startMenu.GREEN + "Enter
your Choice -> ");
        choice = sc.next();
        switch (choice){
            case "1":
                ITMNO =
validateINT.validateInteger(startMenu.G
REEN + "\nEnter Item Code No.: " +
startMenu.RESET);

                System.out.print(startMenu.GREEN +
"Enter Item Description: " +
startMenu.RESET);
                ITMDESC = sc.next();
                QTY =
validateINT.validateInteger(startMenu.G
REEN + "Enter Quantity: " +
startMenu.RESET);
                UPRICE =
validateDouble.validateDouble(startMen
u.GREEN + "Enter Unit Price: " +
startMenu.RESET);
                DISC =
validateDouble.validateDouble(startMen

```

```

u.GREEN + "Enter Discount: " +
startMenu.RESET);
                Product product = new
Product(ITMNO,QTY,UPRICE,DISC,ITM
DESC);
                list.offer(product);

                System.out.println(startMenu.GREEN +
"Enqueued:" + startMenu.RESET +
product);
                break;
            case "2":

                System.out.println(startMenu.GREEN +
"Dequeued:" + startMenu.RESET +
list.poll());
                break;
            case "3":
                if(list.peek() != null){

                    System.out.println(list.peek());
                }else{

                    System.out.println(startMenu.RED_BOL
D + "Queue is Empty." +
startMenu.RESET);
                }
                break;
            case "4":
                display(list);
                break;
            case "5":

                System.out.println(startMenu.GREEN +
"Size: " + startMenu.RESET +
list.size());
                break;
            case "0":

                openQueueMenu.openMenu();

```

```

        default:

System.out.println(startMenu.RED_BOLD + "\nInvalid Choice Selected." +
startMenu.RESET);
    }
}

private static void
display(Queue<Product> queue) {
    if (queue.isEmpty()) {

System.out.println(startMenu.RED_BOLD + "Queue is empty." +
startMenu.RESET);
        return;
    }

System.out.print(startMenu.GREEN +
"Product Information: " +
startMenu.RESET);
    for (Product element : queue) {
        System.out.print(element);
    }
    System.out.println();
}
}

```

openQueueLinkedList

```

import java.util.Scanner;

public class openQueueLinkedList {
    public static void openMenu(){
        LinkedListQueue list = new
LinkedListQueue();
        Scanner sc = new
Scanner(System.in);
        String ITMDESC, choice, ITMNO,
QTY, UPRICE, DISC;
        boolean loop = true;

```

```

while (loop){

System.out.println(startMenu.GREEN_BOLD + "\nQueue Linked List: " +
startMenu.RESET);

System.out.println("-----
-----");

System.out.print(startMenu.GREEN +
"[1]" + startMenu.RESET + " Enqueue
Product\n" +
        startMenu.GREEN + "[2]" +
startMenu.RESET + " Dequeue
Product\n" +
        startMenu.GREEN + "[3]" +
startMenu.RESET + " Peek Product\n" +
        startMenu.GREEN + "[4]" +
startMenu.RESET + " Display
Product/s\n" +
        startMenu.GREEN + "[5]" +
startMenu.RESET + " List Size\n" +
        startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Queue
Data Structure\n" +
        startMenu.GREEN + "Enter
your Choice -> " + startMenu.RESET);
        choice = sc.next();
        switch (choice){
            case "1":

ITMNO =
validateINT.validateInteger(startMenu.G
REEN + "\nEnter Item Code No.: " +
startMenu.RESET);

System.out.print(startMenu.GREEN +
"Enter Item Description: " +
startMenu.RESET);
            ITMDESC = sc.next();

```

```

                                QTY =
validateINT.validateInteger(startMenu.GREEN + "Enter Quantity: " +
startMenu.RESET);

                                UPRICE =
validateDouble.validateDouble(startMenu.GREEN + "Enter Unit Price: " +
startMenu.RESET);

                                DISC =
validateDouble.validateDouble(startMenu.GREEN + "Enter Discount: " +
startMenu.RESET);

list.enqueue(ITMNO,QTY,UPRICE,DISC,ITMDESC);

        break;
    case "2":
        list.dequeue();
        break;
    case "3":
        list.peek();
        break;
    case "4":
        list.display();
        break;
    case "5":

System.out.println(startMenu.GREEN +
"Size: " + startMenu.RESET +
list.size());

        break;
    case "0":

openStackMenu.openMenu();

        default:

System.out.println(startMenu.RED_BOLD + "\nInvalid Choice Selected." +
startMenu.RESET);

    }

```

```

    }
}

openQueueMenu
import java.util.Scanner;

public class openQueueMenu {
    public static void openMenu(){
        Scanner sc = new
Scanner(System.in);
        String choice;
        boolean loop = true;
        while (loop){

System.out.println(startMenu.GREEN_BOLD + "\nQUEUE DATA STRUCTURE"
+ startMenu.RESET);

System.out.println("-----
-----");

System.out.print(startMenu.GREEN +
"[1]" + startMenu.RESET + " Queue
using Array\n" +
        startMenu.GREEN + "[2]" +
startMenu.RESET + " Queue using
Linked List\n" +
        startMenu.GREEN + "[3]" +
startMenu.RESET + " Queue Class in
Java\n" +
        startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Main
Menu\n" +
        startMenu.GREEN + "Enter
your Choice -> " + startMenu.RESET);
        choice = sc.next();
        switch (choice){
            case "1":

```

```

openQueueArray.openMenu();
    case "2":

openQueueLinkedList.openMenu();
    case "3":

openQueueInClass.openMenu();
    case "0":
        mainMenu.openMenu();
    default:

System.out.println(startMenu.RED_BOLD + "\nInvalid Choice Selected." +
startMenu.RESET);
    }
    }
}

```

openSearchAlgorithms

```

import java.util.Scanner;

public class openSearchAlgorithms {
    public static String[][] sampleProducts
= {
        {"1001", "Chocolate", "100",
"39.99", "0.05"},
        {"1002", "Milk", "150", "59.99",
"0.1"},
        {"1003", "Bread", "200", "19.99",
"0.03"},
        {"1004", "Cheese", "120",
"79.99", "0.08"},
        {"1005", "Butter", "75", "49.99",
"0.05"},
        {"1006", "Eggs", "300", "29.99",
"0.02"},
        {"1007", "Juice", "180", "89.99",
"0.07"},

```

```

        {"1008", "Coffee", "50", "149.99",
"0.1"},
        {"1009", "Tea", "90", "119.99",
"0.06"},
        {"1010", "Biscuits", "250",
"25.99", "0.04"},
        {"1011", "Pasta", "160", "34.99",
"0.05"},
        {"1012", "Rice", "220", "49.99",
"0.03"},
        {"1013", "Sugar", "190", "19.99",
"0.02"},
        {"1014", "Salt", "210", "9.99",
"0.01"},
        {"1015", "Pepper", "140",
"24.99", "0.02"},
        {"1016", "Olive Oil", "70",
"199.99", "0.15"},
        {"1017", "Vegetable Oil", "130",
"89.99", "0.1"},
        {"1018", "Soap", "100", "14.99",
"0.03"},
        {"1019", "Shampoo", "60",
"249.99", "0.12"},
        {"1020", "Toothpaste", "110",
"49.99", "0.05"},
        {"1021", "Detergent", "150",
"99.99", "0.08"},
        {"1022", "Dish Soap", "85",
"39.99", "0.04"},
        {"1023", "Tissues", "300",
"19.99", "0.02"},
        {"1024", "Paper Towels", "180",
"34.99", "0.06"},
        {"1025", "Garbage Bags", "90",
"29.99", "0.05"},
        {"1026", "Toilet Paper", "200",
"12.99", "0.03"},
        {"1027", "Cereal", "100", "59.99",
"0.07"},

```

```

        {"1028", "Yogurt", "250", "39.99",
"0.05"},
        {"1029", "Ice Cream", "80",
"99.99", "0.1"},
        {"1030", "Cookies", "120",
"49.99", "0.04"}
    };
    public static void displayProducts() {
        for (int i = 0; i <
sampleProducts.length; i++) {
            System.out.print(
                startMenu.GREEN +
"\nItem Code No.: " + startMenu.RESET
+ sampleProducts[i][0] + " | " +
                startMenu.GREEN +
"Item Description: " + startMenu.RESET
+ sampleProducts[i][1] + " | " +
                startMenu.GREEN +
"Quantity: " + startMenu.RESET +
sampleProducts[i][2] + " | " +
                startMenu.GREEN +
"Unit Price: " + startMenu.RESET +
sampleProducts[i][3] + " | " +
                startMenu.GREEN +
"Discount: " + startMenu.RESET +
sampleProducts[i][4]
            );
        }
        System.out.println();
    }
}

```

```

    public static void openMenu(){
        Scanner sc = new
Scanner(System.in);
        String choice, ans;
        boolean loop = true;
        while (loop){

System.out.println(startMenu.GREEN_B

```

```

OLD + "\nSearch Algorithms " +
startMenu.RESET);

```

```

System.out.println("-----
-----");

```

```

System.out.print(startMenu.GREEN +
"[1]" + startMenu.RESET + " Linear
Search\n" +
                startMenu.GREEN + "[2]" +
startMenu.RESET + " Binary Search\n"
+

```

```

                startMenu.GREEN + "[3]" +
startMenu.RESET + " Interpolation
Search\n" +

```

```

                startMenu.GREEN + "[4]" +
startMenu.RESET + " Display Sample
Records\n" +

```

```

                startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Search
and Sort Menu\n" +

```

```

                startMenu.GREEN + "Enter
your Choice -> " + startMenu.RESET);

```

```

        choice = sc.next();

```

```

        switch (choice){

```

```

            case "1":

```

```

                ans =

```

```

validateINT.validateInteger(startMenu.G
REEN + "Enter Item Code No.: " +
startMenu.RESET);

```

```

searchAlgorithms.linearSearch(ans,sam
pleProducts);

```

```

        break;

```

```

        case "2":

```

```

                ans =

```

```

validateINT.validateInteger(startMenu.G
REEN + "Enter Item Code No.: " +
startMenu.RESET);

```

```

searchAlgorithms.binarySearch(ans,sampleProducts);
    break;
    case "3":
        ans =
validateINT.validateInteger(startMenu.GREEN + "Enter Item Code No.: " +
startMenu.RESET);

searchAlgorithms.interpolationSearch(a
ns,sampleProducts);
    break;
    case "4":
        displayProducts();
        break;
    case "0":

openSearchAndSortMenu.openMenu();
    break;
    default:

System.out.println(startMenu.RED_BOLD + "\nInvalid Choice Selected." +
startMenu.RESET);
    }
    }
    }
}

```

openSearchAndSortMenu

```

import java.util.Scanner;

public class openSearchAndSortMenu {
    public static void openMenu(){
        Scanner sc = new
Scanner(System.in);
        String choice;
        boolean loop = true;
        while (loop){

```

```

System.out.println(startMenu.GREEN_BOLD + "\nSearch And Sort Menu " +
startMenu.RESET);

```

```

System.out.println("-----
-----");

```

```

System.out.print(startMenu.GREEN + "[1]" + startMenu.RESET + " Search
Algorithms\n" +
        startMenu.GREEN + "[2]" +
startMenu.RESET + " Sorting
Algorithms\n" +
        startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Main
Menu\n" +

```

```

        startMenu.GREEN + "Enter
your Choice -> " + startMenu.RESET);
        choice = sc.next();
        switch (choice){
            case "1":

```

```

openSearchAlgorithms.openMenu();
        break;
        case "2":

```

```

openSortAlgorithms.openMenu();
        break;
        case "0":
            mainMenu.openMenu();
            break;
        default:

```

```

System.out.println(startMenu.RED_BOLD + "\nInvalid Choice Selected." +
startMenu.RESET);
    }
    }
    }
}

```

```
}
```

openSortAlgorithms

```
import java.util.Scanner;
```

```
public class openSortAlgorithms {  
    public static String[][] sampleProducts  
= {  
        {"1001", "Chocolate", "100",  
"39.99", "0.05"},  
        {"1002", "Milk", "150", "59.99",  
"0.1"},  
        {"1003", "Bread", "200", "19.99",  
"0.03"},  
        {"1004", "Cheese", "120",  
"79.99", "0.08"},  
        {"1005", "Butter", "75", "49.99",  
"0.05"},  
        {"1006", "Eggs", "300", "29.99",  
"0.02"},  
        {"1007", "Juice", "180", "89.99",  
"0.07"},  
        {"1008", "Coffee", "50", "149.99",  
"0.1"},  
        {"1009", "Tea", "90", "119.99",  
"0.06"},  
        {"1010", "Biscuits", "250",  
"25.99", "0.04"},  
        {"1011", "Pasta", "160", "34.99",  
"0.05"},  
        {"1012", "Rice", "220", "49.99",  
"0.03"},  
        {"1013", "Sugar", "190", "19.99",  
"0.02"},  
        {"1014", "Salt", "210", "9.99",  
"0.01"},  
        {"1015", "Pepper", "140",  
"24.99", "0.02"},  
        {"1016", "Olive Oil", "70",  
"199.99", "0.15"},
```

```
        {"1017", "Vegetable Oil", "130",  
"89.99", "0.1"},  
        {"1018", "Soap", "100", "14.99",  
"0.03"},  
        {"1019", "Shampoo", "60",  
"249.99", "0.12"},  
        {"1020", "Toothpaste", "110",  
"49.99", "0.05"},  
        {"1021", "Detergent", "150",  
"99.99", "0.08"},  
        {"1022", "Dish Soap", "85",  
"39.99", "0.04"},  
        {"1023", "Tissues", "300",  
"19.99", "0.02"},  
        {"1024", "Paper Towels", "180",  
"34.99", "0.06"},  
        {"1025", "Garbage Bags", "90",  
"29.99", "0.05"},  
        {"1026", "Toilet Paper", "200",  
"12.99", "0.03"},  
        {"1027", "Cereal", "100", "59.99",  
"0.07"},  
        {"1028", "Yogurt", "250", "39.99",  
"0.05"},  
        {"1029", "Ice Cream", "80",  
"99.99", "0.1"},  
        {"1030", "Cookies", "120",  
"49.99", "0.04"}  
    };
```

```
    public static String[][] products = {  
        {"1001", "Chocolate", "100",  
"39.99", "0.05"},  
        {"1002", "Milk", "150", "59.99",  
"0.1"},  
        {"1003", "Bread", "200", "19.99",  
"0.03"},  
        {"1004", "Cheese", "120",  
"79.99", "0.08"},
```

```

        {"1005", "Butter", "75", "49.99",
"0.05"},
        {"1006", "Eggs", "300", "29.99",
"0.02"},
        {"1007", "Juice", "180", "89.99",
"0.07"},
        {"1008", "Coffee", "50", "149.99",
"0.1"},
        {"1009", "Tea", "90", "119.99",
"0.06"},
        {"1010", "Biscuits", "250",
"25.99", "0.04"},
        {"1011", "Pasta", "160", "34.99",
"0.05"},
        {"1012", "Rice", "220", "49.99",
"0.03"},
        {"1013", "Sugar", "190", "19.99",
"0.02"},
        {"1014", "Salt", "210", "9.99",
"0.01"},
        {"1015", "Pepper", "140",
"24.99", "0.02"},
        {"1016", "Olive Oil", "70",
"199.99", "0.15"},
        {"1017", "Vegetable Oil", "130",
"89.99", "0.1"},
        {"1018", "Soap", "100", "14.99",
"0.03"},
        {"1019", "Shampoo", "60",
"249.99", "0.12"},
        {"1020", "Toothpaste", "110",
"49.99", "0.05"},
        {"1021", "Detergent", "150",
"99.99", "0.08"},
        {"1022", "Dish Soap", "85",
"39.99", "0.04"},
        {"1023", "Tissues", "300",
"19.99", "0.02"},
        {"1024", "Paper Towels", "180",
"34.99", "0.06"},

```

```

        {"1025", "Garbage Bags", "90",
"29.99", "0.05"},
        {"1026", "Toilet Paper", "200",
"12.99", "0.03"},
        {"1027", "Cereal", "100", "59.99",
"0.07"},
        {"1028", "Yogurt", "250", "39.99",
"0.05"},
        {"1029", "Ice Cream", "80",
"99.99", "0.1"},
        {"1030", "Cookies", "120",
"49.99", "0.04"}
    };

```

```

        public static void
displayProducts(String[][] array) {
    for (int i = 0; i < array.length; i++) {
        System.out.print(
            startMenu.GREEN +
            "\nItem Code No.: " + startMenu.RESET
            + array[i][0] + " | " +
            startMenu.GREEN +
            "Item Description: " + startMenu.RESET
            + array[i][1] + " | " +
            startMenu.GREEN +
            "Quantity: " + startMenu.RESET +
            array[i][2] + " | " +
            startMenu.GREEN +
            "Unit Price: " + startMenu.RESET +
            array[i][3] + " | " +
            startMenu.GREEN +
            "Discount: " + startMenu.RESET +
            array[i][4]
        );
    }
    System.out.println();
}

```

```

    public static void openMenu(){

```

```

        Scanner sc = new
Scanner(System.in);
        String choice, ans;
        boolean loop = true;
        while (loop){

System.out.println(startMenu.GREEN_B
OLD + "\nSort Algorithms " +
startMenu.RESET);

System.out.println("-----
-----");

System.out.print(startMenu.GREEN +
"[1]" + startMenu.RESET + " Sort from
Highest to Lowest Price\n" +
        startMenu.GREEN + "[2]" +
startMenu.RESET + " Sort from Lowest
to Highest Price\n" +
        startMenu.GREEN + "[3]" +
startMenu.RESET + " Display Sample
Records\n" +
        startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Search
and Sort Menu\n" +
        startMenu.GREEN + "Enter
your Choice -> " + startMenu.RESET);
        choice = sc.next();
        switch (choice){
            case "1":

openSortHighestLowest.openMenu();
                break;
            case "2":

openSortLowestHighest.openMenu();
                break;
            case "3":
                displayProducts(products);
                break;

```

```

        case "0":

openSearchAndSortMenu.openMenu();
                break;
            default:

System.out.println(startMenu.RED_BOLD + "\nInvalid Choice Selected." +
startMenu.RESET);
        }
    }
}

```

openSortHighestLowest

```

import java.util.Scanner;

public class openSortHighestLowest {
    public static void openMenu(){
        Scanner sc = new
Scanner(System.in);
        String choice, ans;
        String[][] temp;
        boolean loop = true;
        while (loop){

System.out.println(startMenu.GREEN_B
OLD + "\nSort from Highest to Lowest "
+ startMenu.RESET);

System.out.println("-----
-----");

System.out.print(startMenu.GREEN +
"[1]" + startMenu.RESET + " Bubble
Sort\n" +
        startMenu.GREEN + "[2]" +
startMenu.RESET + " Selection Sort\n"
+

```

```

        startMenu.GREEN + "[3]" +
startMenu.RESET + " Insertion Sort\n" +
        startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Search
and Sort Menu\n" +
        startMenu.GREEN + "Enter
your Choice -> " + startMenu.RESET);
        choice = sc.next();
        switch (choice){
            case "1":

                temp =
openSortAlgorithms.sampleProducts;
                temp =
sortAlgorithms.bubbleSortHL(temp);

openSortAlgorithms.displayProducts(te
mp);

System.out.println(startMenu.GREEN +
"Steps Taken: " + startMenu.RESET +
sortAlgorithms.counter);
                break;
            case "2":

                temp =
openSortAlgorithms.sampleProducts;
                temp =
sortAlgorithms.selectionSortHL(temp);

openSortAlgorithms.displayProducts(te
mp);

System.out.println(startMenu.GREEN +
"Steps Taken: " + startMenu.RESET +
sortAlgorithms.counter);
                break;
            case "3":

                temp =
openSortAlgorithms.sampleProducts;
                temp =
sortAlgorithms.insertionSortHL(temp);

```

```

openSortAlgorithms.displayProducts(te
mp);

```

```

System.out.println(startMenu.GREEN +
"Steps Taken: " + startMenu.RESET +
sortAlgorithms.counter);
                break;
            case "0":

```

```

openSortAlgorithms.openMenu();
                break;
            default:

```

```

System.out.println(startMenu.RED_BOL
D + "\nInvalid Choice Selected." +
startMenu.RESET);
        }
    }
}
}

```

openSortLowestHighest

```

import java.util.Scanner;

```

```

public class openSortLowestHighest {
    public static void openMenu(){
        Scanner sc = new
Scanner(System.in);
        String choice, ans;
        String[][] temp;
        boolean loop = true;
        while (loop){

```

```

System.out.println(startMenu.GREEN_B
OLD + "\nSort from Lowest to Highest "
+ startMenu.RESET);

```

```

System.out.println("-=====
=====");

```

```
System.out.print(startMenu.GREEN +
"[1]" + startMenu.RESET + " Bubble
Sort\n" +
```

```
startMenu.GREEN + "[2]" +
startMenu.RESET + " Selection Sort\n"
+
```

```
startMenu.GREEN + "[3]" +
startMenu.RESET + " Insertion Sort\n" +
startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Search
and Sort Menu\n" +
```

```
startMenu.GREEN + "Enter
your Choice -> " + startMenu.RESET);
```

```
choice = sc.next();
```

```
switch (choice){
```

```
case "1":
```

```
temp =
```

```
openSortAlgorithms.sampleProducts;
```

```
temp =
```

```
sortAlgorithms.bubbleSortLH(temp);
```

```
openSortAlgorithms.displayProducts(te
mp);
```

```
System.out.println(startMenu.GREEN +
"Steps Taken: " + startMenu.RESET +
sortAlgorithms.counter);
```

```
break;
```

```
case "2":
```

```
temp =
```

```
openSortAlgorithms.sampleProducts;
```

```
temp =
```

```
sortAlgorithms.selectionSortLH(temp);
```

```
openSortAlgorithms.displayProducts(te
mp);
```

```
System.out.println(startMenu.GREEN +
```

```
"Steps Taken: " + startMenu.RESET +
sortAlgorithms.counter);
```

```
break;
```

```
case "3":
```

```
temp =
```

```
openSortAlgorithms.sampleProducts;
```

```
temp =
```

```
sortAlgorithms.insertionSortLH(temp);
```

```
openSortAlgorithms.displayProducts(te
mp);
```

```
System.out.println(startMenu.GREEN +
"Steps Taken: " + startMenu.RESET +
sortAlgorithms.counter);
```

```
break;
```

```
case "0":
```

```
openSortAlgorithms.openMenu();
```

```
break;
```

```
default:
```

```
System.out.println(startMenu.RED_BOL
D + "\nInvalid Choice Selected." +
startMenu.RESET);
```

```
}
```

```
}
```

```
}
```

```
}
```

openStackArray

```
import java.util.Scanner;
```

```
public class openStackArray {
    public static void openMenu(){
        String ITMDESC, choice, ITMNO,
QTY, UPRICE, DISC, ans;
        boolean loop = true;
        Scanner sc = new
Scanner(System.in);
```

```

                                ans =
validateINT.validateInteger(startMenu.GREEN + "Array Size: " +
startMenu.RESET);
                                StackArray list = new
StackArray(Integer.parseInt(ans));
                                while (loop){

System.out.println(startMenu.GREEN_BOLD + "\nStack Array " +
startMenu.RESET);

System.out.println("-----
-----");

System.out.print(startMenu.GREEN + "[1]" + startMenu.RESET + " Push
Product\n" +
                                startMenu.GREEN + "[2]" +
startMenu.RESET + " Pop Product\n" +
                                startMenu.GREEN + "[3]" +
startMenu.RESET + " Peek Product\n" +
                                startMenu.GREEN + "[4]" +
startMenu.RESET + " Display
Product/s\n" +
                                startMenu.GREEN + "[5]" +
startMenu.RESET + " List Size\n" +
                                startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Stack
Data Structure\n" +
                                startMenu.GREEN + "Enter
your Choice -> " + startMenu.RESET);
                                choice = sc.next();
                                switch (choice){
                                    case "1":
                                        if(list.isFull()){

System.out.println(startMenu.RED_BOLD + "Stack Overflow! Cannot add more
records." + startMenu.RESET);

```

```

                                }else{
                                ITMNO =
validateINT.validateInteger(startMenu.GREEN + "\nEnter Item Code No.: " +
startMenu.RESET);

System.out.print(startMenu.GREEN + "Enter Item Description: " +
startMenu.RESET);
                                ITMDESC = sc.next();
                                QTY =
validateINT.validateInteger(startMenu.GREEN + "Enter Quantity: " +
startMenu.RESET);

                                UPRICE =
validateDouble.validateDouble(startMenu.GREEN + "Enter Unit Price: " +
startMenu.RESET);

                                DISC =
validateDouble.validateDouble(startMenu.GREEN + "Enter Discount: " +
startMenu.RESET);

                                list.push(ITMNO,QTY,UPRICE,DISC,ITMDESC);

                                break;
                                }
                                case "2":
                                    list.pop();
                                    break;
                                case "3":
                                    list.peek();
                                    break;
                                case "4":
                                    list.display();
                                    break;
                                case "5":

System.out.println(startMenu.GREEN +

```

```

"Size: " + startMenu.RESET +
list.size());
        break;
        case "0":

openStackMenu.openMenu();
        default:

System.out.println(startMenu.RED_BOLD + "\nInvalid Choice Selected." +
startMenu.RESET);
    }
}
}
}

```

openStackInClass

```

import java.util.EmptyStackException;
import java.util.Scanner;
import java.util.Stack;

public class openStackInClass {
    public static void openMenu(){
        String ITMDESC, choice, ITMNO,
QTY, UPRICE, DISC;
        boolean loop = true;
        Scanner sc = new
Scanner(System.in);
        Stack<Product> list = new
Stack<>();
        while (loop){

System.out.println(startMenu.GREEN_BOLD + "\nStack In Class " +
startMenu.RESET);

System.out.println("-----
-----");

System.out.print(startMenu.GREEN +

```

```

"[1]" + startMenu.RESET + " Push
Product\n" +
        startMenu.GREEN + "[2]" +
startMenu.RESET + " Pop Product\n" +
        startMenu.GREEN + "[3]" +
startMenu.RESET + " Peek Product\n" +
        startMenu.GREEN + "[4]" +
startMenu.RESET + " Display
Product/s\n" +
        startMenu.GREEN + "[5]" +
startMenu.RESET + " List Size\n" +
        startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Stack
Data Structure\n" +
        startMenu.GREEN + "Enter
your Choice -> " + startMenu.RESET);
        choice = sc.next();
        switch (choice){
            case "1":
                ITMNO =
validateINT.validateInteger(startMenu.G
REEN + "\nEnter Item Code No.: " +
startMenu.RESET);

System.out.print(startMenu.GREEN +
"Enter Item Description: " +
startMenu.RESET);
                ITMDESC = sc.next();
                QTY =
validateINT.validateInteger(startMenu.G
REEN + "Enter Quantity: " +
startMenu.RESET);
                UPRICE =
validateDouble.validateDouble(startMen
u.GREEN + "Enter Unit Price: " +
startMenu.RESET);
                DISC =
validateDouble.validateDouble(startMen
u.GREEN + "Enter Discount: " +
startMenu.RESET);

```

```

        Product product = new
Product(ITMNO,QTY,UPRICE,DISC,ITM
DESC);
        list.push(product);

System.out.println(startMenu.GREEN +
"Pushed:" + startMenu.RESET +
product);
        break;
    case "2":
        try {

System.out.println(startMenu.GREEN +
"Popped: " + startMenu.RESET +
list.pop());

        } catch
(EmptyStackException e) {

System.out.println(startMenu.RED_BOLD
D + "Stack is Empty." +
startMenu.RESET);
        }
        break;
    case "3":
        try {

System.out.print(list.peek());

        } catch
(EmptyStackException e) {

System.out.println(startMenu.RED_BOLD
D + "Stack is Empty." +
startMenu.RESET);
        }
        break;
    case "4":

System.out.println(startMenu.GREEN +
"Stack: " + startMenu.RESET + list);
        break;

```

```

        case "5":

System.out.println(startMenu.GREEN +
"Size: " + startMenu.RESET +
list.size());
        break;
    case "0":

openQueueMenu.openMenu();
        default:

System.out.println(startMenu.RED_BOLD
D + "\nInvalid Choice Selected." +
startMenu.RESET);
        }
    }
}

openStackLinkedList
import java.util.Scanner;

public class openStackLinkedList {
    public static void openMenu(){
        StackLinkedList list = new
StackLinkedList();
        Scanner sc = new
Scanner(System.in);
        String ITMDESC, choice, ITMNO,
QTY, UPRICE, DISC;
        boolean loop = true;
        while (loop){

System.out.println(startMenu.GREEN_B
OLD + "\nStack Linked List " +
startMenu.RESET);

System.out.println("-----
-----");

```

```

System.out.print(startMenu.GREEN +
"[1]" + startMenu.RESET + " Push
Product\n" +
        startMenu.GREEN + "[2]" +
startMenu.RESET + " Pop Product\n" +
        startMenu.GREEN + "[3]" +
startMenu.RESET + " Peek Product\n" +
        startMenu.GREEN + "[4]" +
startMenu.RESET + " Display
Product/s\n" +
        startMenu.GREEN + "[5]" +
startMenu.RESET + " List Size\n" +
        startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Stack
Data Structure\n" +
        startMenu.GREEN + "Enter
your Choice -> " + startMenu.RESET);
        choice = sc.next();
        switch (choice){
            case "1":
                ITMNO =
validateINT.validateInteger(startMenu.G
REEN + "\nEnter Item Code No.: " +
startMenu.RESET);

                System.out.print(startMenu.GREEN +
"Enter Item Description: " +
startMenu.RESET);
                ITMDESC = sc.next();

                QTY =
validateINT.validateInteger(startMenu.G
REEN + "Enter Quantity: " +
startMenu.RESET);

                UPRICE =
validateDouble.validateDouble(startMen
u.GREEN + "Enter Unit Price: " +
startMenu.RESET);

                DISC =
validateDouble.validateDouble(startMen

```

```

u.GREEN + "Enter Discount: " +
startMenu.RESET);

```

```

list.push(ITMNO,QTY,UPRICE,DISC,IT
MDESC);

```

```

        break;
        case "2":
            list.pop();
            break;
        case "3":
            list.peek();
            break;
        case "4":
            list.display();
            break;
        case "5":

```

```

System.out.println(startMenu.GREEN +
"Size: " + startMenu.RESET +
list.size());
        break;
        case "0":

```

```

openStackMenu.openMenu();
        default:

```

```

System.out.println(startMenu.RED_BOL
D + "\nInvalid Choice Selected." +
startMenu.RESET);

```

```

        }
    }
}

```

openStackMenu

```

import java.util.Scanner;

```

```

public class openStackMenu {
    public static void openMenu(){

```

```

        Scanner sc = new
Scanner(System.in);
        String choice;
        boolean loop = true;
        while (loop){

System.out.println(startMenu.GREEN_B
OLD + "\nSTACK DATA STRUCTURE"
+ startMenu.RESET);

System.out.println("-----
-----");

System.out.print(startMenu.GREEN +
"[1]" + startMenu.RESET + " Stack using
Array\n" +
        startMenu.GREEN + "[2]" +
startMenu.RESET + " Stack using
Linked List\n" +
        startMenu.GREEN + "[3]" +
startMenu.RESET + " Stack Class in
Java\n" +
        startMenu.GREEN + "[0]" +
startMenu.RESET + " Back to Main
Menu\n" +
        startMenu.GREEN + "Enter
your Choice -> " + startMenu.RESET);
        choice = sc.next();
        switch (choice){
            case "1":

openStackArray.openMenu();
            case "2":

openStackLinkedList.openMenu();
            case "3":

openStackInClass.openMenu();
            case "0":
                mainMenu.openMenu();

```

default:

```

System.out.println(startMenu.RED_BOL
D + "\nInvalid Choice Selected." +
startMenu.RESET);
        }
    }
}
}

```

productData

```

import java.util.LinkedList;
import java.util.Scanner;

public class productData {
    int ITMNO;
    String ITMDESC;
    int QTY;
    double UPRICE;
    double DISC;

    public productData(int ITMNO, String
ITMDESC, int QTY, double UPRICE,
double DISC) {
        this.ITMNO = ITMNO;
        this.ITMDESC = ITMDESC;
        this.QTY = QTY;
        this.UPRICE = UPRICE;
        this.DISC = DISC;
    }

    public static void openMenu() {
        LinkedList<productData>
productDataLists = new LinkedList<>();
        Scanner input = new
Scanner(System.in);

        boolean running = true;
        while (running) {
            System.out.println();

```

```

System.out.println(startMenu.GREEN_BOLD + "Linked List in Class Menu");
        System.out.println("\nInventory Management System" + startMenu.RESET); //di ko alam name

```

```

System.out.println(startMenu.GREEN + "1." + startMenu.RESET + " Add Product");

```

```

System.out.println(startMenu.GREEN + "2." + startMenu.RESET + " Display Products");

```

```

System.out.println(startMenu.GREEN + "3." + startMenu.RESET + " Remove a Product");

```

```

System.out.println(startMenu.GREEN + "0." + startMenu.RESET + " Back to Linked List Menu");

```

```

System.out.print(startMenu.GREEN + "Choose an action: " + startMenu.RESET);
        String choice = input.next();

```

```

        switch (choice) {
            case "1":
                int ITMNO = Integer.parseInt(validateINT.validateInteger(startMenu.GREEN + "Enter Item Code No.: " + startMenu.RESET));

```

```

System.out.print(startMenu.GREEN + "Enter Item Description: " + startMenu.RESET);

```

```

        String ITMDESC = input.next();

```

```

                int QTY = Integer.parseInt(validateINT.validateInteger(startMenu.GREEN + "Enter Quantity: " + startMenu.RESET));

```

```

                double UPRICE = Double.parseDouble(validateDouble.validateDouble(startMenu.GREEN + "Enter Unit Price: " + startMenu.RESET));

```

```

                double DISC = Double.parseDouble(validateDouble.validateDouble(startMenu.GREEN + "Enter Discount: " + startMenu.RESET));

```

```

                productDataLists.add(new productData(ITMNO, ITMDESC, QTY, UPRICE, DISC));
                break;
            case "2":

```

```

                if (productDataLists.isEmpty()) {

```

```

                    System.out.println(startMenu.RED_BOLD + "\n No products available." + startMenu.RESET);
                } else {

```

```

                    for (productData product : productDataLists) {
                        product.display();
                    }
                }
                break;

```

```

            case "3":
                int deleteID = Integer.parseInt(validateINT.validateInteger(startMenu.GREEN + "Enter Product Code Number to delete: " + startMenu.RESET));

```

```

                boolean found = false;

```

```

                for (int i = 0; i <
productDataLists.size(); i++) {
                    if
(productDataLists.get(i).ITMNO    ==
(deleteID)) {

productDataLists.remove(i);
                found = true;

System.out.println(startMenu.GREEN +
"Successfully removed a product with ID
" + startMenu.RESET + deleteID);
                break;
            }
        }
        if (!found) {

System.out.println(startMenu.RED_BOLD
D + "Product with ID " +
startMenu.RESET + deleteID +
startMenu.RED_BOLD + " not found." +
startMenu.RESET);
        }
        break;

        case "0":

openLinkedListMenu.openMenu();
            break;

        default:

System.out.println(startMenu.RED_BOLD
D + "Invalid input, please try again." +
startMenu.RESET);
        }
    }
}

```

```

public void display() {

System.out.println(startMenu.GREEN +
"Item Code No.: " + startMenu.RESET +
ITMNO +
        "," + startMenu.GREEN + "
Item Description: " + startMenu.RESET
+ ITMDESC +
        "," + startMenu.GREEN + "
Quantity: " + startMenu.RESET + QTY +
        "," + startMenu.GREEN + "
Unit Price: " + startMenu.RESET +
UPRICE +
        "," + startMenu.GREEN + "
Discount: " + startMenu.RESET +
DISC);
    }
}

```

queueNode

```

public class queueNode {
    Product data;
    queueNode next;
    queueNode(Product data) {
        this.data = data;
        next = null;
    }
}

```

searchAlgorithms

```

import java.util.*;

public class searchAlgorithms {

    public static void linearSearch(String
key, String[][] data){
        boolean isFound = false;
        int steps = 0;
        for (int i = 0; i < data.length; i++) {
            steps++;

```

```

        if
        (data[i][0].equalsIgnoreCase(key)) {

System.out.print(startMenu.GREEN +
"Given key " + startMenu.RESET + key
+ startMenu.GREEN + " is found at
index " + startMenu.RESET + i);

System.out.println(startMenu.GREEN +
"\nSteps Taken: " + startMenu.RESET +
steps);

        isFound = true;
        break;
    }
}
if (!isFound) {

System.out.println(startMenu.RED_BOLD
D + "Given key " + startMenu.RESET +
key + startMenu.RED_BOLD + " is not
present in the array." +
startMenu.RESET);
}
}

```

```

    public static void binarySearch(String
key, String[][] data) {
        int first = 0, steps = 0;
        int last = data.length - 1;
        boolean isFound = false;

        while (first <= last) {
            steps++;
            int mid = (first + last) / 2;

            if (data[mid][0].equals(key)) {

System.out.print(startMenu.GREEN +
"Given key " + startMenu.RESET + key

```

```

+ startMenu.GREEN + " is found at
index " + startMenu.RESET + mid);

System.out.println(startMenu.GREEN +
"\nSteps Taken: " + startMenu.RESET +
steps);

        isFound = true;
        break;
    } else if
        (data[mid][0].compareTo(key) < 0) {
            first = mid + 1;
        } else {
            last = mid - 1;
        }
    }

    if (!isFound) {

System.out.println(startMenu.RED_BOLD
D + "Given key " + startMenu.RESET +
key + startMenu.RED_BOLD + " is not
present in the array." +
startMenu.RESET);
}
}

```

```

        public static void
interpolationSearch(String key, String[][]
data) {
            int low = 0, steps = 0, high =
data.length - 1;
            boolean isFound = false;

            while (low <= high &&
key.compareTo(data[low][0]) >= 0 &&
key.compareTo(data[high][0]) <= 0) {
                steps++;

                int mid = low +
((Integer.parseInt(key)
-

```

```

Integer.parseInt(data[low][0])) * (high -
low))
/
(Integer.parseInt(data[high][0])
-
Integer.parseInt(data[low][0]));

    if (data[mid][0].equals(key)) {

System.out.print(startMenu.GREEN +
"Given key " + startMenu.RESET + key
+ startMenu.GREEN + " is found at
index " + startMenu.RESET + mid);

System.out.println(startMenu.GREEN +
"\nSteps Taken: " + startMenu.RESET +
steps);
        ifFound = true;
        break;
    } else if
(data[mid][0].compareTo(key) < 0) {
        low = mid + 1; // Move to the
right part
    } else {
        high = mid - 1; // Move to the
left part
    }
}

    if (!isFound) {

System.out.println(startMenu.RED_BOLD + "Given key " + startMenu.RESET +
key + startMenu.RED_BOLD + " is not
present in the array." +
startMenu.RESET);
    }
}
}

```

sortAlgorithms

```

public class sortAlgorithms {

    public static int counter;

    public static String[][]
bubbleSortHL(String[][] data){
        counter = 0;
        for (int i = 0; i < data.length - 1; i++)
        {
            for (int j = 0; j < data.length - i -
1; j++) {
                counter++;

                if
(Double.parseDouble(data[j][3]) <
Double.parseDouble(data[j + 1][3])) {
                    String[] temp = data[j];
                    data[j] = data[j + 1];
                    data[j + 1] = temp;
                }
            }
        }
        return data;
    }

    public static String[][]
selectionSortHL(String[][] data){
        counter = 0;
        for (int i = 0; i < data.length - 1; i++)
        {
            int smallest = i;
            for (int j = i + 1; j < data.length;
j++) {
                if
(Double.parseDouble(data[j][3]) >
Double.parseDouble(data[smallest][3]))
                {
                    smallest = j;
                }
            }
        }
    }
}

```

```

        String[] temp = data[i];
        data[i] = data[smallest];
        data[smallest] = temp;
        counter++;
    }
    return data;
}

```

```

        public static String[][]
insertionSortHL(String[][] data){
    counter = 0;
    // Insertion Sort Algorithm
    for (int i = 1; i < data.length; i++) {
        counter++;
        String[] key = data[i];
        int j = i - 1;
        // Shift elements of the sorted
        segment to make room for the key
        while (j >= 0 &&
Double.parseDouble(data[j][3]) <
Double.parseDouble(key[3])) {
            data[j + 1] = data[j];
            j--;
        }
        data[j + 1] = key;
    }
    return data;
}

```

```

        public static String[][]
bubbleSortLH(String[][] data){
    counter = 0;
    for (int i = 0; i < data.length - 1; i++)
    {
        for (int j = 0; j < data.length - i -
1; j++) {
            counter++;
            if
(Double.parseDouble(data[j][3]) >
Double.parseDouble(data[j + 1][3])) {

```

```

        String[] temp = data[j];
        data[j] = data[j + 1];
        data[j + 1] = temp;
    }
}
return data;
}

```

```

        public static String[][]
selectionSortLH(String[][] data){
    counter = 0;
    for (int i = 0; i < data.length - 1; i++)
    {
        int smallest = i;
        for (int j = i + 1; j < data.length;
j++) {
            if
(Double.parseDouble(data[j][3]) <
Double.parseDouble(data[smallest][3]))
            {
                smallest = j;
            }
        }
        String[] temp = data[i];
        data[i] = data[smallest];
        data[smallest] = temp;
        counter++;
    }
    return data;
}

```

```

        public static String[][]
insertionSortLH(String[][] data){
    counter = 0;
    // Insertion Sort Algorithm
    for (int i = 1; i < data.length; i++) {
        counter++;
        String[] key = data[i];
        int j = i - 1;

```

```

        // Shift elements of the sorted
        segment to make room for the key
        while (j >= 0 &&
Double.parseDouble(data[j][3]) >
Double.parseDouble(key[3])) {
            data[j + 1] = data[j];
            j--;
        }
        data[j + 1] = key;
    }
    return data;
}
}

```

tictactoe_Application

```
import java.util.Scanner;
```

```

public class tictactoe_Application {
    static String[][] arr = { { "1", "2", "3" }, {
"4", "5", "6" }, { "7", "8", "9" } };
    static String[][] arr2 = { { "1", "2", "3" },
{ "4", "5", "6" }, { "7", "8", "9" } };
    static int ctr = 0, flag, row, col, draw =
0, win = 0;
    static String player = "X";
    static Scanner sc = new
Scanner(System.in);
    static int move;

    //tictactoe
    public static void displayboard() {
        for (int x = 0; x < 3; x++) {
            for (int y = 0; y < 3; y++) {
                System.out.print(arr[x][y] + "
");
            }
            System.out.println();
        }
    }
    public static void resetBoard() {

```

```

        for (int x = 0; x < 3; x++) {
            for (int y = 0; y < 3; y++) {
                arr[x][y] = arr2[x][y];
            }
        }
    }
    public static void MakeMove() {
        if (player.equals("X")) {
            move = sc.nextInt();

        } else if (player.equals("O")) {
            move = sc.nextInt();

        }
        switch (move) {
            case 1:
                row = 0;
                col = 0;
                break;
            case 2:
                row = 0;
                col = 1;
                break;
            case 3:
                row = 0;
                col = 2;
                break;
            case 4:
                row = 1;
                col = 0;
                break;
            case 5:
                row = 1;
                col = 1;
                break;
            case 6:
                row = 1;
                col = 2;
                break;

```

```

        case 7:
            row = 2;
            col = 0;
            break;
        case 8:
            row = 2;
            col = 1;
            break;
        case 9:
            row = 2;
            col = 2;
    }
}

public static void placeMove() {
    if (!(arr[row][col]).equals("X") &&
!(arr[row][col]).equals("O") && player ==
"X") {
        arr[row][col] = player;
        player = "O";
        ctr++;
    } else if (!(arr[row][col]).equals("X")
&& !(arr[row][col]).equals("O") && player
== "O") {
        arr[row][col] = player;
        player = "X";
        ctr++;
    } else {
        System.out.println("INVALID
MOVE");
    }
}

public static void ifGameOver() {

    for (int x = 0; x < 3 && flag == 0;
x++) {
        // for straight patterns
        if (arr[x][0] == arr[x][1] &&
arr[x][1] == arr[x][2]) {

```

```

            flag = 1;
            win = 1;

            } else if (arr[0][x] == arr[1][x] &&
arr[1][x] == arr[2][x]) {
                flag = 1;
                win = 1;
            }
            // for diagonal patterns
            if (arr[0][0] == arr[1][1] &&
arr[1][1] == arr[2][2]) {
                flag = 1;
                win = 1;
            }
            if (arr[0][2] == arr[1][1] &&
arr[1][1] == arr[2][0]) {
                flag = 1;
                win = 1;
            }
        }
        if (ctr > 8) {
            draw = 1;
            flag = 1;
        }
    }

    public static void ticTacToe() {
        win = 0;
        flag = 0;
        ctr = 0;
        draw = 0;
        System.out.println("WELCOME TO
TIC TAC TOE!!!!");

        System.out.println("-----
");

        while (win == 0) {
            System.out.println("TURN = " +
player);

```

```

System.out.println("-----
");
        displayboard();

System.out.println("-----
");

        MakeMove();
        placeMove();
        ifGameOver();
    }
    displayboard();
    if (player == "X" && win == 1) {
        System.out.println("PLAYER O
WINS");
    } else if (player == "O" && win == 1)
    {
        System.out.println("PLAYER X
WINS");
    } else {
        System.out.println("ITS A
DRAW!!!!");
    }
    resetBoard();
}
}
}

```

validateDouble

```
import java.util.Scanner;

public class validateDouble {
    public static String
    validateDouble(String question){
        Scanner sc = new
        Scanner(System.in);
        String ans;
        double checker = 0;
        boolean validated;
        while (true) {
```

```

        validated = true;
        System.out.print(question);
        ans = sc.next();
        try {
            checker = Double.parseDouble(ans);
        } catch (Exception e) {
            System.out.println(startMenu.RED_BOLD + "\nInvalid input." + startMenu.RESET + " Please enter a double value.");
            validated = false;
        }
        if (validated){
            return Double.toString(checker);
        }
    }
}

```

validateINT

```
import java.util.*;

public class validateINT {
    public static String
    validateInteger(String question){
        Scanner sc = new
        Scanner(System.in);
        String ans;
        int checker = 0;
        boolean validated;
        while (true) {
            validated = true;
            System.out.print(question);
            ans = sc.next();
            try {
                Integer.parseInt(ans);
                checker =
```

```

        } catch (Exception e) {
            System.out.println(startMenu.RED_BOLD + "\nInvalid input. " +
startMenu.RESET + "Please enter an integer.");
            validated = false;
        }

        if (validated){
            Integer.toString(checker);
        }
    }
}
return

```

Program Listing (Java Classes, Methods, and Objects)

1. ArrayQueue

Methods:

- isFull()
- isEmpty()
- enqueue(String ITMNO, String QTY, String UPRICE, String DISC, String ITMDESC)
- dequeue()
- peek()
- display()
- size()

2. HashTable

Class:

2.1 Data

Methods:

- add(int id, String ITMNO, String QTY, String UPRICE, String DISC, String ITMDESC)
- display()
- delete(int id)

3. Item

Methods:

- addItem()
- displayItem()
- deleteItem()

- editDetails()

4. LinkedListQueue

Methods:

- isEmpty()
- enqueue(String ITMNO, String QTY, String UPRICE, String DISC, String ITMDESC)
- dequeue()
- peek()
- display()
- size()

5. Main

Method:

- MainMenu()

6. OneDimensional

Methods:

- mainMenu()
- addItem()
- displayItem()
- editDetails()
- deleteItem()

7. Product

Object:

- Product

Method:

- toString()

8. SinglyProduct

Object:

- SinglyProduct

Classes:

8.1 SinglyNode

8.2 SinglyLinkedList

Methods:

- newSinglyProduct(SinglyProduct product)
- display()
- delete(int ITMNO)

Methods:

- toString()
- singlyLinkedListMenu()

9. StackArray

Methods:

- isFull()
- push(String ITMNO, String QTY, String UPRICE, String DISC, String ITMDESC)
- pop()
- peek()
- isEmpty()
- size()
- display()

10. StackLinkedList

Methods:

- push(String ITMNO, String QTY, String UPRICE, String DISC, String ITMDESC)
- pop()
- peek()
- isEmpty()
- size()
- display()

11. StackNode

12. TwoDimensional

Methods:

- mainMenu()
- addItem()
- displayItem()
- editDetails()
- deleteItem()

13. arrApplication

Method:

- mainMenu()

14. connectFour_Application

Methods:

- resetBoard()
- showBoard()
- MakeMove()
- PlaceMove()
- checkWinner()
- connectFour()

15. doublyProductData

Object:

- doublyProductData

Classes:

15.1 DoublyNode

15.2 DoublyLinkedList

Methods:

- newProduct(doublyProductData product)
- display()
- displayReverse()
- delete(int ITMNO)

Methods:

- toString()
- doublyLinkedListMenu()

16. mainMenu

Method:

- openMenu()

17. openGroupMembersMenu

Method:

- openMenu()

18. openHashFunctionMenu

Method:

- openMenu()

19. openHashTableMenu

Method:

- openMenu()

20. openLinkedListMenu

Method:

- openMenu()

21. openQueueArray

Method:

- openMenu()

22. openQueueInClass

Methods:

- openMenu()
- display(Queue<Product> queue)

23. openQueueLinkedList

Method:

- openMenu()

24. openQueueMenu

Method:

- openMenu()

25. openSearchAlgorithms

Methods:

- displayProducts()
- openMenu()

26. openSearchAndSortMenu

Method:

- openMenu()

27. openSortAlgorithms

Methods:

- displayProducts(String[] array)
- openMenu()

28. openSortHighestLowest

Method:

- openMenu()

29. openSortLowestHighest

Method:

- openMenu()

30. openStackArray

Method:

- openMenu()

31. openStackInClass

Method:

- openMenu()

32. openStackLinkedList

Method:

- openMenu()

33. openStackMenu

Method:

- openMenu()

34. productData

Object:

- productData

Methods:

- openMenu()
- display()

35. queueNode

36. searchAlgorithms

Methods:

- linearSearch(String key, String[] data)
- binarySearch(String key, String[] data)
- interpolationSearch(String key, String[] data)

37. sortAlgorithms

Methods:

- bubbleSortHL(String[][] data)
- selectionSortHL(String[][] data)
- insertionSortHL(String[][] data)
- bubbleSortLH(String[][] data)
- selectionSortLH(String[][] data)
- insertionSortLH(String[][] data)

38. startMenu

Methods:

- openMenu()
- main(String[] args)

39. tictactoe_Application

Methods:

- displayboard()
- resetBoard()

- MakeMove()
- placeMove()
- ifGameOver()
- ticTacToe()
- displayboard()

40. validateDouble

Method:

- validateDouble(String question)

41. validateINT

Method:

- validateInteger(String question)

Group Status Report

Group Leader : Oleriana, Mico Laurenz D.

Members : Amojilar, Rogell B.,
Dela Cruz, Maria Kylee A.
Gonzalez, Rheanna V.
Manzano, Elohim Lee L.
San Pedro, Jerome M.
Villanueva, Mikaela Marie A.

Member Name	Assigned Task	Completed	Partial
Oleriana, Mico Laurenz D.		✓	
Amojilar, Rogell B.		✓	
Dela Cruz, Maria Kylee A.		✓	
Gonzalez, Rheanna V.		✓	
Manzano, Elohim Lee L.		✓	
San Pedro, Jerome M.		✓	
Villanueva, Mikaela Marie A.		✓	

Attendance (Meeting):

Member Name	November 18, 2024	December 2, 2024	December 9, 2024	Summary
-------------	----------------------	---------------------	---------------------	---------

Oleriana, Mico Laurenz D.	Present	Present	Present	P=3 A=0
Amojilar, Rogell B.	Present	Present	Present	P=3 A=0
Dela Cruz, Maria Kylee A.	Present	Present	Present	P=3 A=0
Gonzalez, Rheanna V.	Present	Present	Present	P=3 A=0
Manzano, Elohim Lee L.	Present	Present	Present	P=3 A=0
San Pedro, Jerome M.	Present	Present	Present	P=3 A=0
Villanueva, Mikaela Marie A.	Present	Present	Present	P=3 A=0

References

GeeksforGeeks. (2023, August 7). Difference between Arrays and Pointers. GeeksforGeeks. <https://www.geeksforgeeks.org/difference-between-array-and-pointers/>

GeeksforGeeks. (2024, November 26). LinkedList in java. GeeksforGeeks. <https://www.geeksforgeeks.org/linked-list-in-java/>

GeeksforGeeks. (2024, December 6). What is Stack Data Structure? A Complete Tutorial. GeeksforGeeks. <https://www.geeksforgeeks.org/introduction-to-stack-data-structure-and-algorithm-tutorials/>

GeeksforGeeks. (2024, November 18). Queue data structure. GeeksforGeeks. <https://www.geeksforgeeks.org/queue-data-structure/>

GeeksforGeeks. (2024a, November 12). Sorting algorithms. GeeksforGeeks. <https://www.geeksforgeeks.org/sorting-algorithms/>

GeeksforGeeks. (2024a, October 4). Hashtable in Java. GeeksforGeeks.
<https://www.geeksforgeeks.org/hashtable-in-java/>